ADVANCED TRACK

AI 에이전트 개발 고급

LangChain · LangGraph · Deep Agents

주식회사 똑똑한청년들 · 배움 에이아이

PART 01

오리엔테이션

LangChain 기초, LangGraph 심화, Deep Agents 실전 위에 미들웨어·멀티에이전트·메모리·RAG·SQL·데이터 분석·음성·프로덕션을 얹습니다.

고급 트랙은 에이전트를 기능 구현에서 운영 가능한 시스템으로 끌어올립니다

목표는 새로운 API를 많이 아는 것이 아니라 실패·권한·상태·평가를 설계할 수 있게 되는 것입니다

Middleware · Multi-agent · Context/Memory · Agentic RAG · SQL/Data/Voice · Production

이 덱은 고급 트랙 노트북 10개를 따라가는 학습 안내서입니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

05_advanced는 10개 노트북으로 고급 운영 패턴을 묶습니다

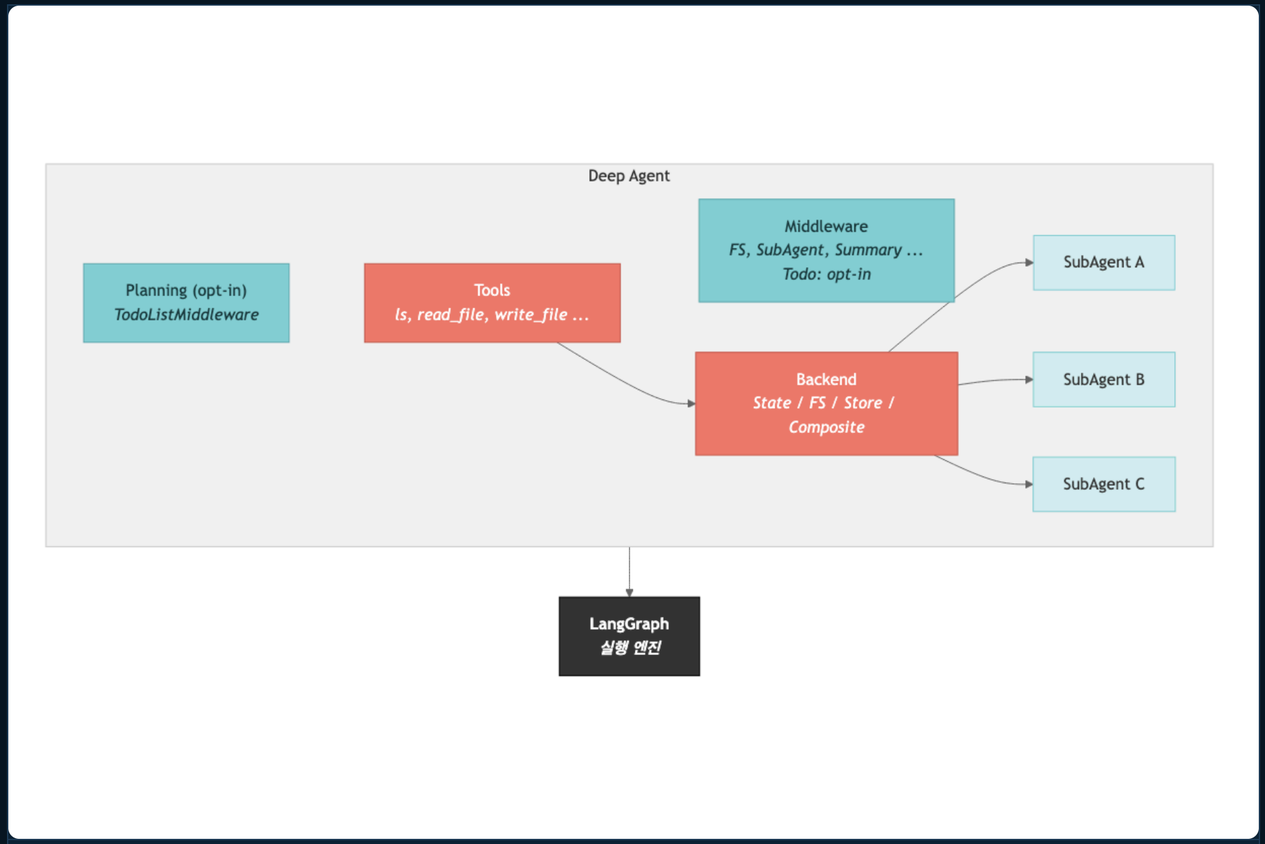
각 노트북은 하나의 운영 문제를 다룹니다

항목	핵심	실전 판단
마이그레이션	v1 패키지·API·상태 스키마 전환	기존 코드를 최신 구조로 옮깁니다
미들웨어	6단계 훅과 wrap_model/tool_call	실행 경계를 제어합니다
멀티에이전트	Subagents, Handoffs, Router	작업 분리와 상태 전환을 설계합니다
컨텍스트/메모리	runtime context, Store, semantic memory	정보 수명을 분리합니다
응용 에이전트	RAG, SQL, 데이터 분석, 음성	현업 데이터 작업으로 확장합니다
프로덕션	테스트, 평가, 배포, 관측성	출시 기준을 세웁니다

© 주식회사 똑똑한청년들 · 배움 에이아이

고급 트랙의 공통 축은 실행 흐름을 제어하고 상태를 회수 가능하게 만드는 것입니다

모든 노트북은 모델 답변이 아니라 실행 경계와 운영 루프를 설명합니다



advanced architecture map

Control

미들웨어와 그래프로 실행 흐름을 제어합니다.

State

체크포인터, Store, runtime context로 정보를 나눕니다.

Evaluation

테스트와 평가로 출시 기준을 검증합니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 02

v1 마이그레이션

레거시 LangChain 코드를 v1 패키지 구조, create_agent, TypedDict state, runtime context, middleware 방식으로 옮깁니다.

실습 · 05_advanced/00_migration.ipynb

마이그레이션 실습은 import, agent 생성, state, middleware를 순서대로 검증합니다

한 번에 전부 바꾸지 말고 실행 표면별로 통과 여부를 확인합니다

체크포인트	확인할 내용	통과 기준
import	langchain-classic 의존 위치 확인	deprecated 경로가 남지 않습니다
agent 생성	create_agent로 모델·도구 연결	기존 질문이 같은 도구를 호출합니다
state	TypedDict 기반 상태 정리	그래프와 타입 오류가 없습니다
middleware	동적 프롬프트와 오류 처리 이동	실패 케이스가 ToolMessage 등으로 복구됩니다

© 주식회사 똑똑한청년들 · 배움 에이아이

마이그레이션 리스크는 조용한 행동 변화에서 발생합니다

코드가 실행돼도 도구 순서나 출력 계약이 바뀌면 운영 회귀입니다

주의할 회귀

- 도구 호출 이름이 바뀌어 기존 평가가 실패합니다
- context 전달 위치가 바뀌어 사용자 권한이 누락됩니다
- Pydantic state를 그대로 두어 LangGraph 호환성이 떨어집니다

완료 기준

- 대표 질문 세트가 이전과 같은 행동을 보입니다
- trace에서 도구 호출과 오류 처리가 확인됩니다

© 주식회사 똑똑한청년들 · 배움 에이아이

마이그레이션은 import 변경보다 실행 모델 변경을 먼저 봐야 합니다

패키지 이름만 바꾸면 코드가 돌아가도 운영 기능을 놓칠 수 있습니다

항목	핵심	실전 판단
패키지	langchain-classic 분리	레거시 chain/agent 경로를 확인합니다
Agent API	create_react_agent 중심에서 create_agent 중심으로 이동	도구·모델·미들웨어 결합 표면이 바뀝니다
State	Pydantic state보다 TypedDict state를 사용	LangGraph 호환성을 확보합니다
Runtime context	config가 아니라 context_schema로 런타임 정보를 주입	도구와 미들웨어가 같은 정보를 읽습니다
Middleware	동적 프롬프트와 도구 오류 처리를 훅으로 이동	프롬프트 분기와 예외 처리를 구조화합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

create_agent는 모델·도구·미들웨어를 한 실행 표면에서 묶습니다

고급 트랙에서는 create_agent를 기본 조립 단위로 보고 LangGraph/Deep Agents로 확장합니다

model

모델 객체 또는 모델 문자열을 전달합니다.

tools

도구는 함수 또는 Tool 객체로 연결합니다.

middleware

동적 프롬프트, HITL, 오류 처리 등을 삽입합니다.

00_migration.ipynb — create_agent

```python
from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-5.4-mini",
    tools=[search_tool, calculator],
    middleware=[dynamic_prompt_middleware],
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "분석해줘"}]
})
```

© 주식회사 똑똑한청년들 · 배움 에이아이

마이그레이션 검수는 동작 여부보다 의미 보존 여부를 봐야 합니다

기존 코드가 실행되는 것과 기존 권한·상태·출력 계약이 유지되는 것은 다릅니다

- **코드 검수**
  - import 경로와 deprecated API를 확인합니다
  - state schema가 TypedDict 기준으로 정리됐는지 봅니다
  - configurable 값이 context_schema로 옮겨졌는지 봅니다
- **행동 검수**
  - 도구 호출 순서가 유지되는지 확인합니다
  - 오류 처리와 재시도 정책이 middleware로 이동했는지 봅니다
  - 테스트 질문에서 이전 답변 계약이 유지되는지 검증합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 03

미들웨어

모델 호출 전후, 도구 호출 전후, 에이전트 전후 훅으로 실행 흐름을 제어합니다.

실습 · 05_advanced/01_middleware.ipynb

미들웨어 실습은 훅 위치와 반환 타입을 함께 확인해야 합니다

훅은 실행 위치에 따라 허용되는 입력과 반환이 다릅니다

체크포인트	확인할 내용	통과 기준
before_agent	입력 검증과 초기 상태	차단 또는 상태 업데이트가 명확합니다
wrap_model_call	모델 호출 감싸기	handler 호출 전후 정책이 분리됩니다
wrap_tool_call	도구 오류와 권한	ToolMessage 또는 Command 처리가 맞습니다
after_agent	최종 응답 검사	로그와 평가 정보가 남습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

미들웨어 리스크는 너무 많은 정책을 한 훅에 넣을 때 커집니다

정책을 섞으면 실패 원인과 테스트 범위가 불분명해집니다

분리 원칙

- 입력 차단은 before_agent에 둡니다
- 모델 선택은 wrap_model_call에 둡니다
- 도구 오류 복구는 wrap_tool_call에 둡니다

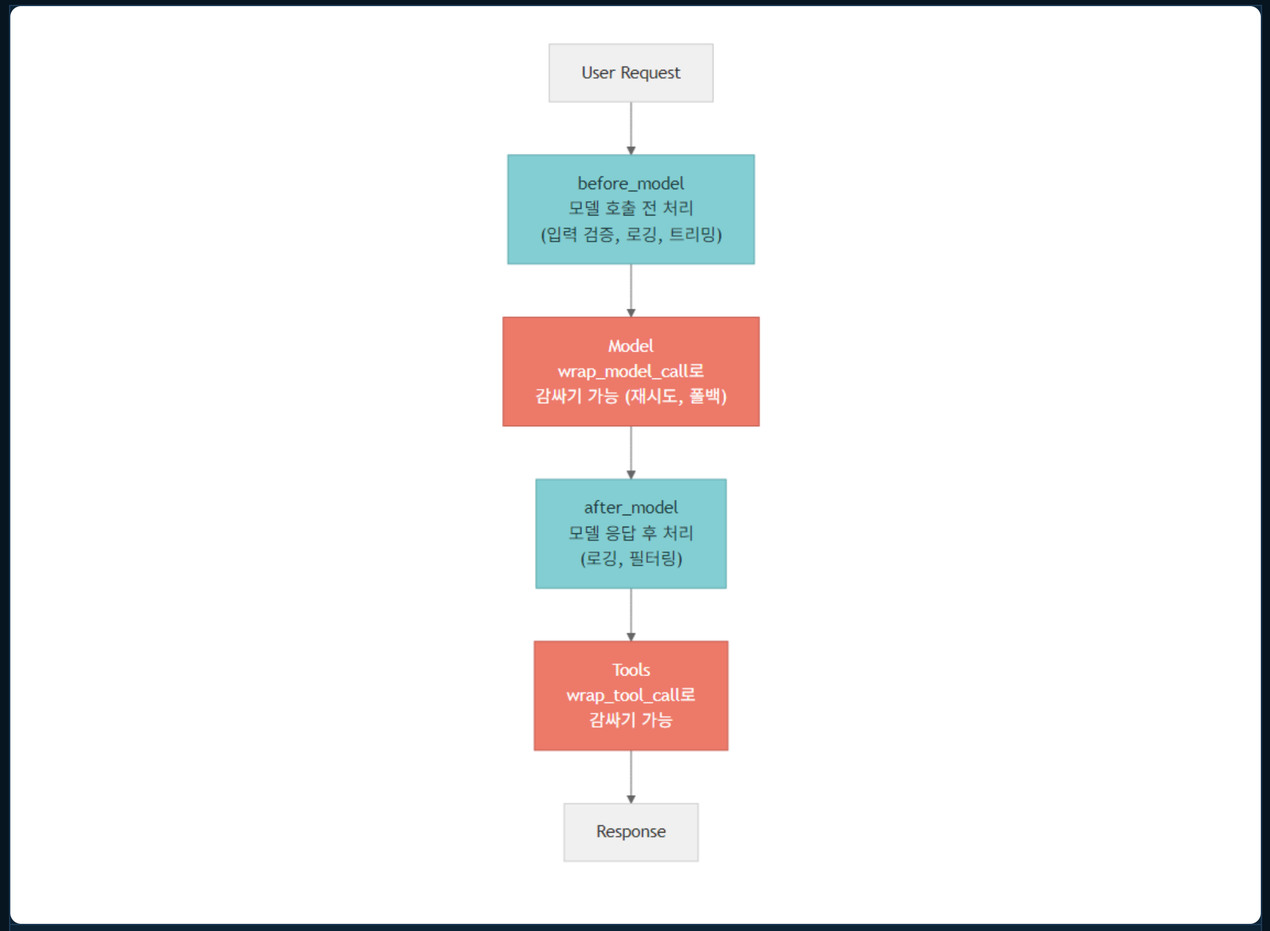
테스트 기준

- 허용 케이스와 차단 케이스를 모두 둡니다
- 예외가 사용자에게 그대로 노출되지 않아야 합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

미들웨어는 에이전트 실행 파이프라인의 각 지점에 정책을 삽입합니다

고급 에이전트는 프롬프트보다 미들웨어로 운영 정책을 관리하는 경우가 많습니다



middleware pipeline

before_agent

입력 검증과 초기 컨텍스트 조립을 수행합니다.

wrap_model_call

모델 호출을 감싸 모델·프롬프트·응답을 제어합니다.

wrap_tool_call

도구 오류, 재시도, 권한 확인을 처리합니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

6단계 훅은 정책을 어디에 넣을지 정하는 지도입니다

같은 정책도 위치가 다르면 의미가 달라집니다

항목	핵심	실전 판단
before_agent	에이전트 실행 전	입력 검증, PII 차단
before_model	모델 호출 전	동적 프롬프트, 컨텍스트 압축
wrap_model_call	모델 호출 감싸기	모델 라우팅, fallback, 비용 제어
wrap_tool_call	도구 호출 감싸기	권한, 오류 복구, 재시도
after_model	모델 응답 후	출력 검사, 도구 호출 정책 확인
after_agent	최종 응답 후	감사 로그, 결과 검증

© 주식회사 똑똑한청년들 · 배움 에이아이

wrap_tool_call은 도구 오류를 모델이 읽을 수 있는 관측값으로 바꿉니다

예외를 그대로 터뜨리면 대화가 끊기지만 ToolMessage로 바꾸면 에이전트가 복구할 수 있습니다

request

도구 호출 이름과 인자를 포함합니다.

handler

실제 도구 실행을 호출합니다.

ToolMessage

오류를 도구 관측값으로 반환합니다.

01_middleware.ipynb

wrap_tool_call

```python
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage

@wrap_tool_call
def handle_tool_errors(request, handler):
    try:
        return handler(request)
    except Exception as exc:
        return ToolMessage(
            content=f"도구 실행 실패: {exc}",
            tool_call_id=request.tool_call["id"],
        )
```

© 주식회사 똑똑한청년들 · 배움 에이아이

HumanInTheLoopMiddleware는 위험 도구를 실행 전에 멈춥니다

승인·수정·거절이 필요한 도구는 모델 판단만으로 실행하지 않습니다

항목	핵심	실전 판단
approve	도구 호출을 그대로 실행	낮은 위험의 확인 절차
edit	인자를 사람이 수정해 실행	파일 경로, SQL 조건, 수신자 수정
reject	도구 호출을 실행하지 않음	위험 요청 차단
respond	사람이 직접 사용자 응답 제공	정책 판단이 필요한 경우

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 04

Subagents

전문 에이전트를 도구처럼 호출해 컨텍스트를 격리하고 역할별 도구 권한을 제한합니다.

실습 · 05_advanced/02_multi_agent_subagents.ipynb

Subagents 실습은 역할 경계와 권한 경계를 같이 검수합니다

전문 역할을 나눴다면 호출 조건과 도구 범위도 달라야 합니다

체크포인트	확인할 내용	통과 기준
description	호출 조건 문장	감독자가 역할을 안정적으로 고릅니다
tools	역할별 최소 도구	권한이 불필요하게 넓지 않습니다
output	하위 결과 형식	감독자가 짧게 합성할 수 있습니다
state	히스토리 관리 방식	상속/독립 체크포인터가 의도와 맞습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Subagents 리스크는 위임이 아니라 결과 회수에서 드러납니다

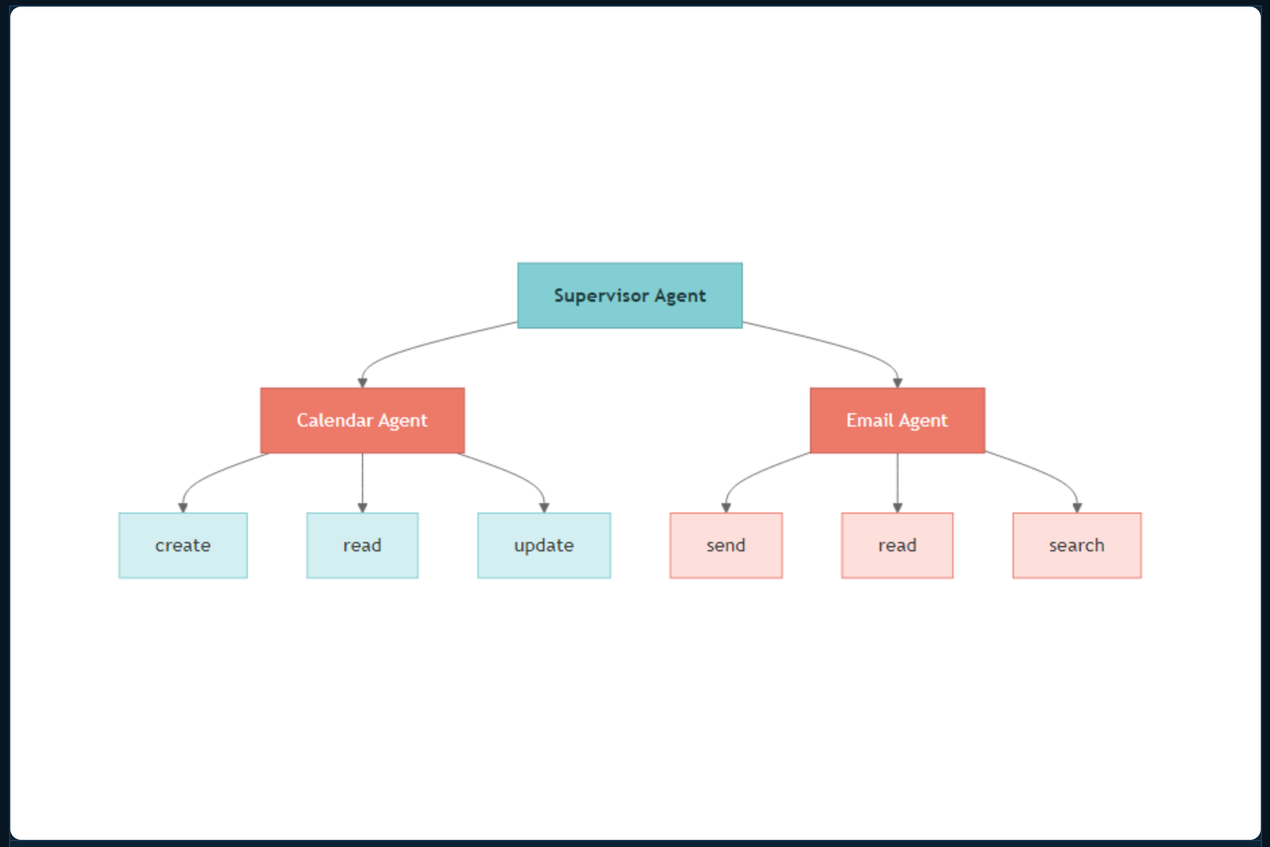
하위 작업 결과가 길거나 모호하면 메인 컨텍스트가 다시 부풀어 오릅니다

- **좋은 회수**
  - 핵심 결론과 출처만 반환합니다
  - 다음 행동이 필요한 경우 명시합니다
- **나쁜 회수**
  - 원문 전체를 그대로 반환합니다
  - 하위 에이전트가 사용자에게 직접 말하는 톤을 유지합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Subagents는 감독자가 전문 에이전트를 도구처럼 호출하는 중앙 집중 패턴입니다

사용자는 감독자와 대화하고, 전문 작업은 별도 컨텍스트에서 처리됩니다



supervisor + subagents

Supervisor

라우팅과 최종 응답을 책임집니다.

Subagent

전문 도구와 프롬프트만 가지고 독립 작업합니다.

Context isolation

긴 하위 작업이 메인 대화를 오염시키지 않습니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

서브에이전트는 역할 분리와 컨텍스트 격리를 동시에 제공합니다

역할을 많이 만드는 것보다 description과 도구 권한을 선명하게 하는 것이 중요합니다

항목	핵심	실전 판단
name	도구 이름처럼 노출되는 식별자	짧고 명확해야 합니다
description	언제 호출할지 판단하는 라우팅 문장	가장 중요한 필드입니다
system_prompt	하위 역할의 행동 계약	출력 형식과 금지 행동을 넣습니다
tools	하위 역할이 쓸 수 있는 도구	최소 권한 원칙을 적용합니다
checkpointer	히스토리 유지 방식	상속과 독립 히스토리를 구분합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

SubAgentMiddleware는 supervisor에게 task 디스패치 도구를 노출합니다

직접 @tool로 감싸는 방식보다 역할 목록을 선언적으로 관리할 수 있습니다

subagents

역할 정의 목록입니다.

middleware

감독자 에이전트에 위임 도구를 주입합니다.

task

모델이 하위 역할에게 작업을 보냅니다.

02_multi_agent_subagents.ipynb SubAgentMiddleware

```python
from langchain.agents import create_agent
from langchain.agents.middleware import SubAgentMiddleware

subagents = [{
    "name": "researcher",
    "description": "자료 조사와 출처 수집이 필요할 때 사용합니다.",
    "system_prompt": "근거 중심 리서처로 답하세요.",
    "tools": [search_tool],
}]

supervisor = create_agent(
    model=model,
    tools=[],
    middleware=[SubAgentMiddleware(subagents=subagents)],
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

서브에이전트 설계의 실패는 대부분 역할 경계가 흐릴 때 생깁니다

각 역할이 같은 도구와 같은 출력 형식을 가지면 분리 효과가 줄어듭니다

- **좋은 분리**
  - 역할마다 입력과 출력이 다릅니다
  - 도구 권한이 역할별로 제한됩니다
  - 감독자는 결과 요약만 받습니다
- **나쁜 분리**
  - 모든 서브에이전트가 같은 일을 할 수 있습니다
  - description이 모호해 라우팅이 흔들립니다
  - 하위 결과가 너무 길어 메인 컨텍스트를 다시 채웁니다

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 05

Handoffs와 Router

상태 기반 순차 전환과 분류 기반 병렬 라우팅을 구분합니다.

실습 · 05_advanced/03_multi_agent_handoffs_router.ipynb

Handoffs와 Router 실습은 전이 메시지와 fan-out 결과를 확인합니다

멀티에이전트 오류는 대부분 상태 전이 또는 결과 합성에서 발생합니다

체크포인트	확인할 내용	통과 기준
Handoff ToolMessage	tool_call_id 매칭	대화 히스토리가 malformed 되지 않습니다
current_step	상태 전이 값	단계별 프롬프트와 도구가 바뀝니다
Send fan-out	워커 호출 목록	필요한 도메인만 병렬 실행됩니다
Reducer	결과 합성	출처와 충돌 내용이 남습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

패턴 선택 리스크는 순차 업무를 병렬화하거나 병렬 검색을 순차화할 때 생깁니다

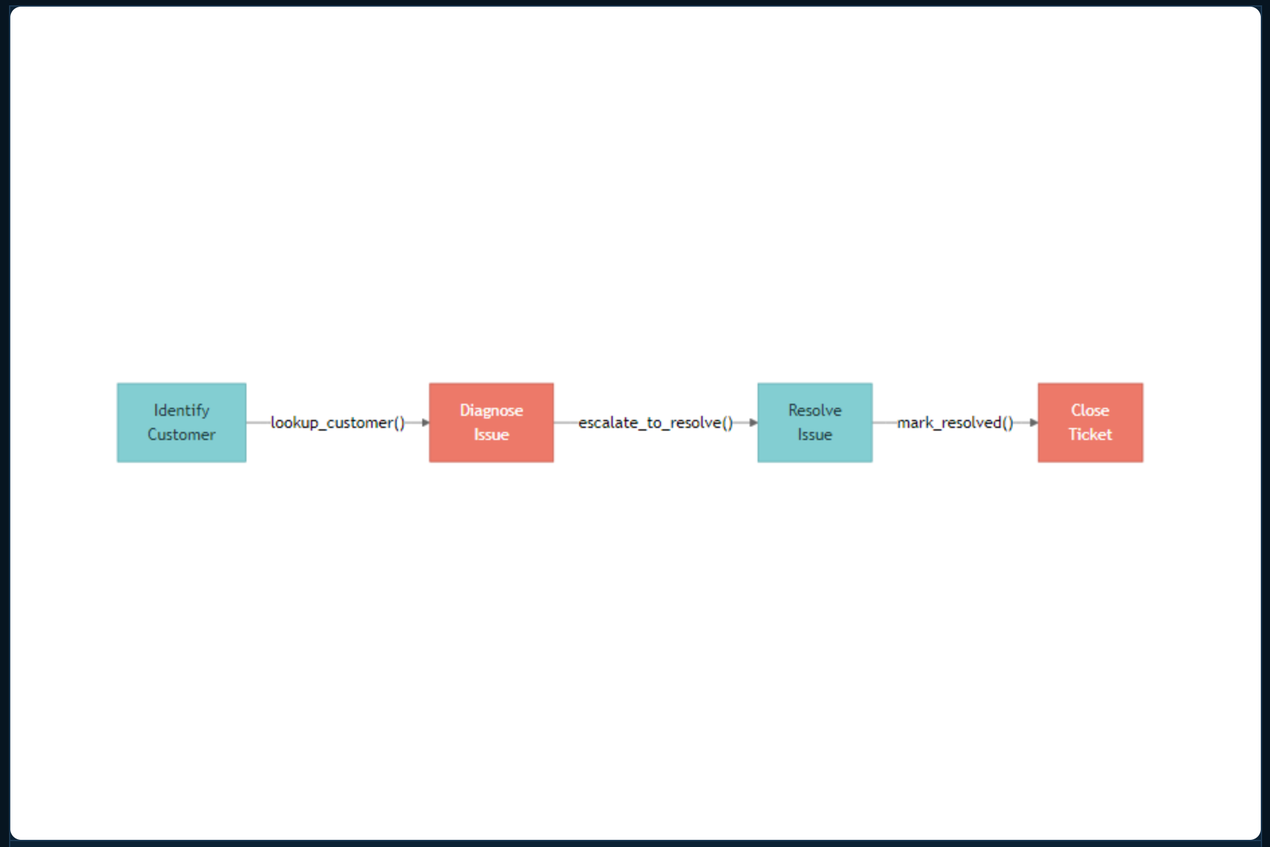
제어 흐름이 업무 흐름과 맞지 않으면 성능보다 사용자 경험이 먼저 무너집니다

- **Handoffs가 맞는 경우**
  - 사용자와 단계별로 직접 대화합니다
  - 앞 단계 정보가 뒤 단계의 전제입니다
- **Router가 맞는 경우**
  - 여러 지식 소스를 동시에 검색합니다
  - 분류 결과에 따라 필요한 워커 수가 달라집니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Handoffs는 단일 에이전트가 상태에 따라 다른 페르소나와 도구를 갖는 패턴입니다

고객 지원처럼 순차 단계가 있는 업무는 current_step 상태로 진행 단계를 관리합니다



handoff state machine

current_step

현재 단계를 추적하는 상태 변수입니다.

Command(update)

핸드오프 도구가 상태를 바꿉니다.

wrap_model_call

단계별 system prompt와 tools를 교체합니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

핸드오프 도구는 Command(update)와 ToolMessage를 함께 반환해야 합니다

도구 호출/응답 쌍이 맞지 않으면 모델 대화 히스토리가 잘못된 형태가 됩니다

runtime

tool_call_id를 얻습니다.

current_step

다음 단계로 상태를 전환합니다.

ToolMessage

핸드오프 ack를 메시지에 남깁니다.

03_multi_agent_handoffs_router.ipynb — handoff tool

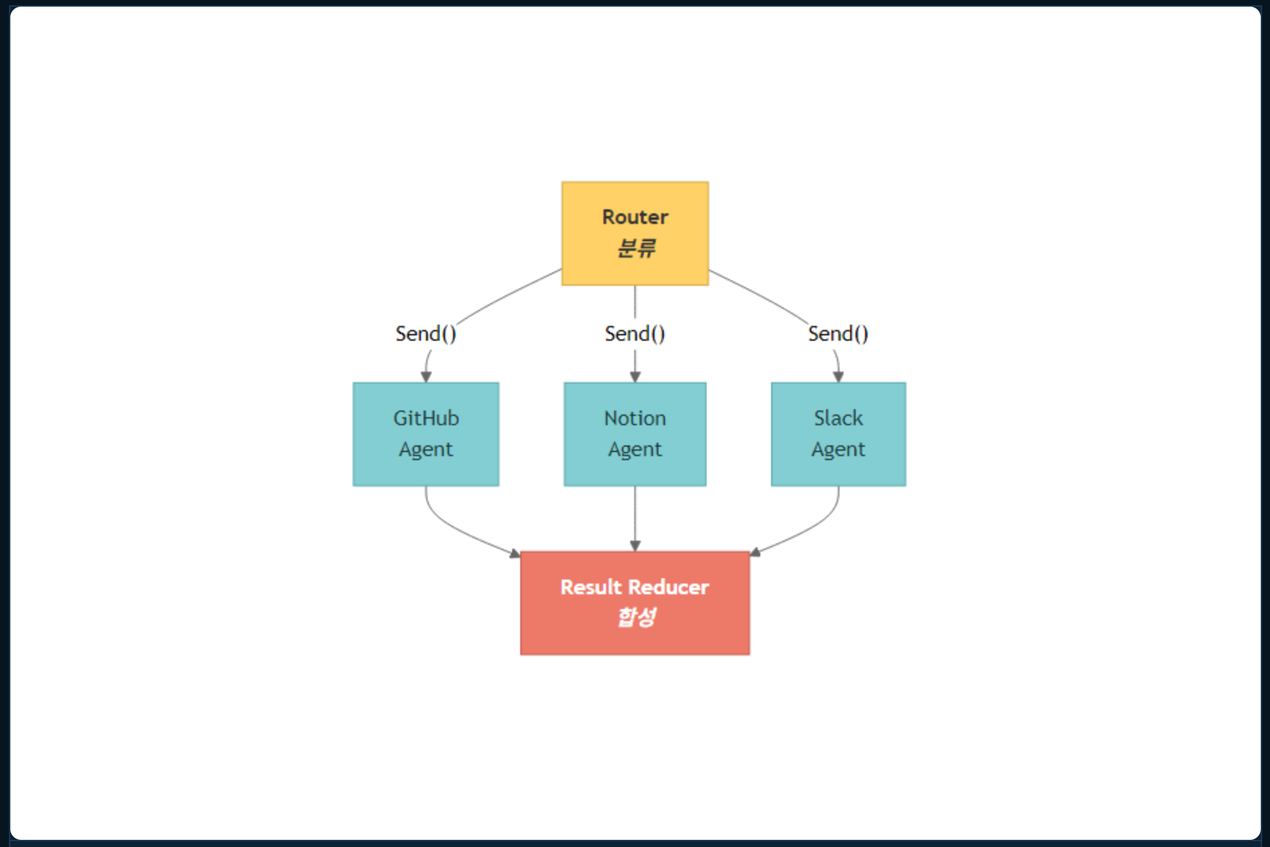
```python
from langchain.tools import tool, ToolRuntime
from langgraph.types import Command
from langchain.messages import ToolMessage

@tool
def transfer_to_resolve(runtime: ToolRuntime[None, SupportState]) -> Command:
    return Command(update={
        "current_step": "resolve_issue",
        "messages": [ToolMessage(
            content="Transferred to resolve step",
            tool_call_id=runtime.tool_call_id,
        )],
    })
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Router는 분류 후 Send로 여러 도메인에 fan-out하고 reducer에서 합성합니다

병렬 지식 검색과 결과 합성이 필요한 경우 Handoffs보다 Router가 적합합니다



router fan-out / fan-in

Classify

질문을 도메인 또는 작업 유형으로 분류합니다.

Send

필요한 워커를 병렬로 실행합니다.

Reducer

여러 결과를 출처와 함께 합성합니다.

Subagents, Handoffs, Router는 제어 주체가 다릅니다

멀티에이전트 패턴은 에이전트 수가 아니라 제어 흐름으로 구분합니다

항목	핵심	실전 판단
Subagents	감독자가 제어	전문 작업 위임
Handoffs	상태가 제어	순차 단계와 사용자 직접 대화
Router	분류 결과가 제어	병렬 검색과 fan-out
Custom	그래프 설계자가 제어	복합 정책과 특수 요구사항

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 06

컨텍스트와 메모리

runtime context, store, semantic memory, progressive disclosure로 정보 수명과 가변성을 분리합니다.

실습 · 05_advanced/04_context_memory.ipynb

컨텍스트 실습은 정보의 수명과 가변성을 먼저 분류합니다

정보가 어디에 있어야 하는지 정해야 비용과 보안 경계가 보입니다

체크포인트	확인할 내용	통과 기준
Runtime context	실행 중 고정 정보	권한·사용자 id가 도구에 주입됩니다
State	현재 실행 중 변화 정보	노드 사이 값이 추적됩니다
Store	스레드 밖 장기 정보	다음 대화에서도 검색됩니다
Prompt	이번 호출에 필요한 지시	토큰 낭비가 없습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

메모리 리스크는 오래 보관할 필요 없는 정보를 장기 저장할 때 커집니다

장기 메모리는 편리하지만 삭제, 권한, 개인정보 책임을 함께 가집니다

저장 전 질문

- 다음 대화에서도 필요한 정보인가요?
- 사용자가 삭제를 요청할 수 있나요?
- 다른 thread나 사용자에게 보이면 안 되나요?

운영 기준

- namespace로 사용자와 조직을 분리합니다
- 민감정보는 저장 전 마스킹하거나 저장하지 않습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

컨텍스트는 가변성과 수명을 기준으로 나눠야 합니다

무엇을 prompt에 넣을지보다 어디에 얼마나 오래 보관할지를 먼저 결정합니다

항목	핵심	실전 판단
Static runtime context	실행 중 바뀌지 않는 설정	사용자 id, 권한, 조직 정보
Dynamic state	그래프 실행 중 바뀌는 상태	messages, 중간 결과
Cross-thread store	스레드 밖에서 공유되는 장기 정보	사용자 프로필, 장기 메모리
Skill/procedure	필요할 때 읽는 절차 지식	작업 규칙, 도구 사용법

© 주식회사 똑똑한청년들 · 배움 에이아이

context_schema는 런타임 정보를 타입 있는 객체로 주입합니다

도구와 미들웨어가 같은 사용자·권한 정보를 일관되게 읽게 만듭니다

dataclass

런타임 컨텍스트 스키마입니다.

context_schema

agent가 받을 context 타입을 지정합니다.

runtime.context

도구 실행 중 컨텍스트를 읽습니다.

04_context_memory.ipynb

context_schema

```python
from dataclasses import dataclass
from langchain.tools import ToolRuntime, tool

@dataclass
class UserContext:
    user_id: str
    role: str

@tool
def lookup_policy(query: str, runtime: ToolRuntime[UserContext]) → str:
    role = runtime.context.role
    return f"{role} 권한으로 정책 검색: {query}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Store는 thread를 넘어 공유되는 장기 메모리를 저장합니다

checkpointer가 실행 이력을 저장한다면 Store는 사용자나 조직 단위의 장기 지식을 저장합니다

namespace

사용자나 도메인별 저장 공간입니다.

put

장기 사실을 저장합니다.

search

필요한 기억을 검색합니다.

04_context_memory.ipynb — Store memory

```
namespace = ("users", user_id, "memories")
store.put(namespace, "preference", {"topic": "LangGraph"})

memories = store.search(namespace, query="관심 주제")
for item in memories:
    print(item.value)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

컨텍스트 엔지니어링은 많이 넣는 기술이 아니라 필요한 정보만 회수하는 기술입니다

정보를 잘못된 층에 넣으면 비용, 누락, 권한 문제가 동시에 생깁니다

- **prompt에 넣을 것**
  - 이번 답변에 반드시 필요한 짧은 지침
  - 모델이 바로 따라야 하는 출력 계약
- **store에 넣을 것**
  - 다음 대화에서도 필요한 장기 사실
  - 사용자별로 검색 가능한 선호와 히스토리
- **state에 넣을 것**
  - 현재 실행 중만 필요한 중간 결과
  - 그래프 노드 사이에 전달되는 작업 상태

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 07

Agentic RAG

검색 결과를 평가하고 질문을 다시 쓰는 그래프형 RAG를 구축합니다.

실습 · 05_advanced/05_agentic_rag.ipynb

Agentic RAG 실습은 검색 품질과 복구 경로를 따로 검증합니다

최종 답변만 보면 검색 실패와 생성 실패를 구분할 수 없습니다

체크포인트	확인할 내용	통과 기준
검색 결과	정답 문서가 상위에 있는지	top-k 안에 기대 근거가 있습니다
관련성 평가	grade_documents 결과	무관한 문서는 답변으로 가지 않습니다
질문 재작성	rewrite_question 효과	재검색 결과가 개선됩니다
최종 답변	출처와 한계	근거 없는 답변이 없습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Agentic RAG 리스크는 재시도가 품질 개선이 아니라 루프가 될 때 발생합니다

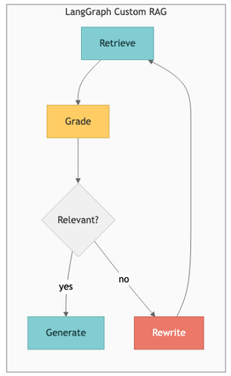
복구 경로에는 항상 횟수 제한과 실패 응답 정책이 필요합니다

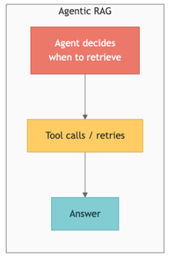
- **제한할 것**
  - 질문 재작성 횟수
  - 검색 top-k와 컨텍스트 길이
  - 모델 호출 총량
- **실패 시 응답**
  - 근거 부족을 명시합니다
  - 추가로 필요한 문서나 질문을 제안합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Agentic RAG는 검색 후 관련성 평가와 질문 재작성을 포함합니다

단순 RAG가 실패하는 지점을 그래프의 복구 경로로 명시합니다







RAG architecture choices

Retrieve

질문에 맞는 문서 후보를 가져옵니다.

Grade

문서가 질문에 답할 수 있는지 평가합니다.

Rewrite

관련성이 낮으면 질문을 검색 친화적으로 다시 씁니다.

retrieve 도구는 content와 artifact를 함께 반환해야 평가와 출처 추적이 가능합니다

모델이 읽을 문자열과 운영자가 추적할 Document 객체를 분리합니다

serialized

모델 입력에 넣는 근거 문자열입니다.

docs

원본 Document 객체입니다.

metadata

출처 표시와 필터링에 사용합니다.

05_agentic_rag.ipynb — retrieve tool

```python
from langchain_core.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    docs = vector_store.similarity_search(query, k=4)
    serialized = "\n\n".join(
        f"출처: {d.metadata.get('source', '?')}\n{d.page_content}"
        for d in docs
    )
    return serialized, docs
```

relevance_router는 관련성 평가 결과를 답변 또는 재검색 경로로 바꿉니다

그래프형 RAG의 핵심은 검색 실패를 최종 답변 전에 감지하는 것입니다

relevant

답변 생성으로 이동합니다.

not relevant

질문 재작성으로 이동합니다.

StateGraph

복구 경로를 실행 그래프에 고정합니다.

05_agentic_rag.ipynb — relevance routing

```python
def relevance_router(state: AgentState):
    if state.get("relevance") == "relevant":
        return "generate_answer"
    return "rewrite_question"

graph = StateGraph(AgentState)
graph.add_node("retrieve", ToolNode([retrieve]))
graph.add_node("grade_documents", grade_documents)
graph.add_node("rewrite_question", rewrite_question)
graph.add_node("generate_answer", generate_answer)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

세 가지 RAG 접근은 제어 가능성과 비용이 다릅니다

복잡한 질문일수록 복구 경로가 필요하지만 호출 비용과 지연이 늘어납니다

항목	핵심	실전 판단
RAG Chain	검색 후 답변 흐름 고정	단순 질의응답
RAG Agent	도구 사용 여부를 모델이 판단	다단계 검색 질문
LangGraph RAG	평가·재작성·답변을 그래프로 고정	운영 복구와 관찰 가능성
Deep Agent RAG	파일·스킬·장기 작업까지 포함	프로젝트형 리서치 작업

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 08

SQL Agent

자연어 질문을 안전한 SQL 조회와 검증 가능한 답변으로 연결합니다.

실습 · 05_advanced/06_sql_agent.ipynb

SQL Agent 실습은 생성 SQL을 최종 답변만큼 중요하게 다룹니다

데이터 기반 답변은 SQL과 결과를 함께 검토해야 합니다

체크포인트	확인할 내용	통과 기준
schema 조회	사용 가능한 테이블 확인	없는 컬럼을 쓰지 않습니다
query 생성	SELECT와 조건 구성	허용 테이블과 LIMIT를 지킵니다
query 실행	결과 행과 오류 확인	오류를 수정 루프로 보냅니다
answer	업무 언어로 해석	쿼리와 결과 근거가 남습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

SQL Agent 리스크는 자연어가 권한 높은 쿼리로 바뀌는 순간 발생합니다

읽기 전용이어도 민감 데이터 대량 조회는 운영 사고가 될 수 있습니다

- **차단할 요청**
  - 개인정보 전체 추출
  - 권한 밖 테이블 조회
  - 쓰기·삭제·스키마 변경
- **완화 방법**
  - 허용 테이블과 행 수를 제한합니다
  - 위험 쿼리는 HITL 승인을 둡니다

© 주식회사 똑똑한청년들 · 배움 에이아이

SQLDatabaseToolkit은 DB 스키마 조회와 SQL 실행 도구를 에이전트에 제공합니다

SQL 에이전트는 먼저 스키마를 읽고, 쿼리를 작성하고, 결과를 해석해야 합니다

SQLDatabase

연결 대상 DB와 테이블 정보를 제공합니다.

Toolkit

스키마 조회와 쿼리 실행 도구를 만듭니다.

Agent

도구를 사용해 자연어 질문에 답합니다.

06_sql_agent.ipynb — SQL toolkit

```
from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import SQLDatabaseToolkit
from langchain.agents import create_agent

db = SQLDatabase.from_uri("sqlite:///Chinook.db")
toolkit = SQLDatabaseToolkit(db=db, llm=model)
agent = create_agent(model=model, tools=toolkit.get_tools())
```

© 주식회사 똑똑한청년들 · 배움 에이아이

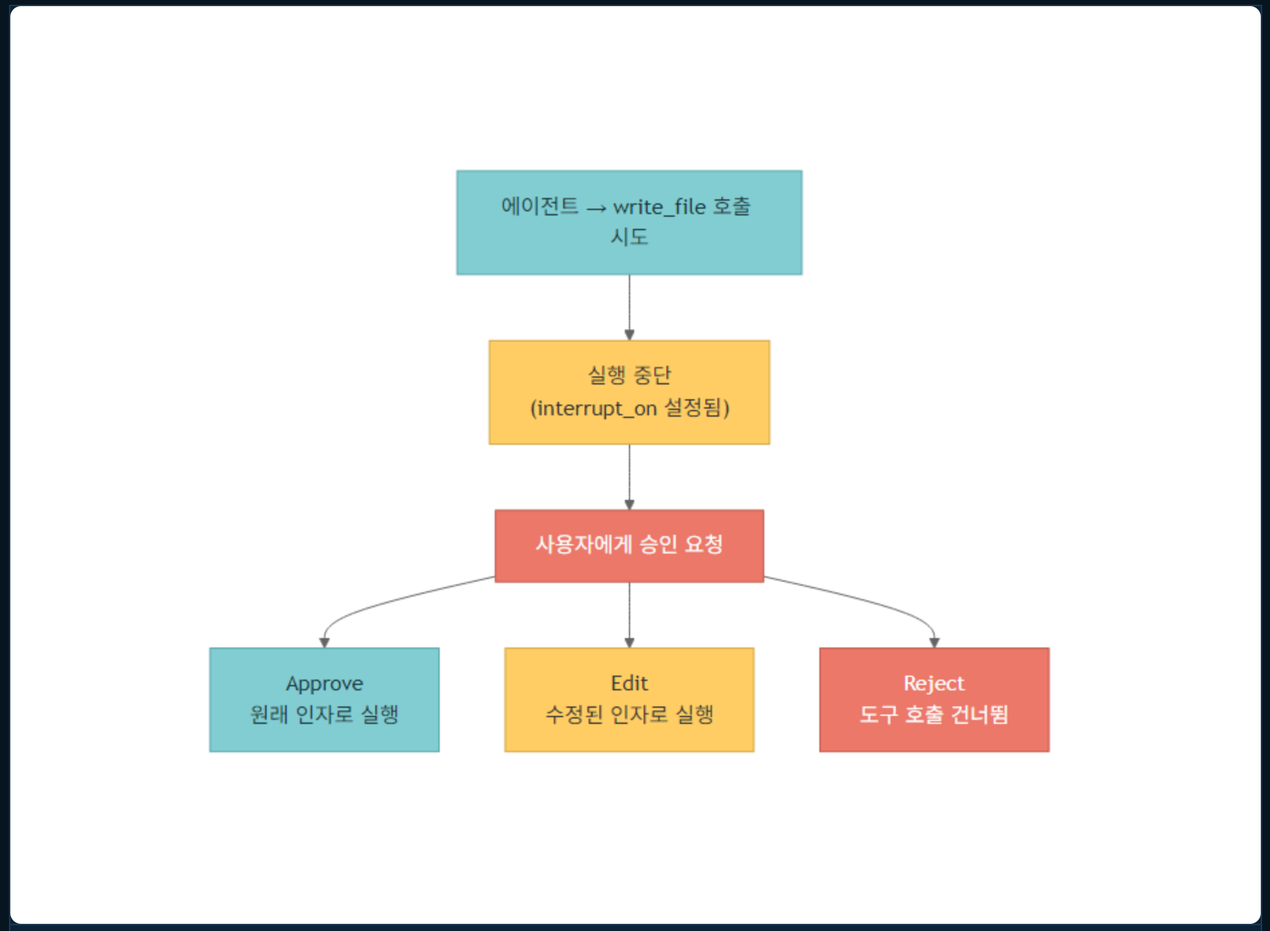
SQL 에이전트 안전 수칙은 조회 범위와 실행 권한을 제한하는 데서 시작합니다

자연어가 SQL이 되는 순간 데이터 삭제와 대량 조회 위험을 함께 봐야 합니다

항목	핵심	실전 판단
읽기 전용	SELECT 중심 권한 사용	쓰기·삭제 차단
테이블 제한	허용 테이블만 노출	민감 테이블 보호
쿼리 검토	실행 전 SQL 확인	HITL 승인
행 수 제한	LIMIT 기본 적용	비용과 정보 노출 제한
감사 로그	질문·SQL·결과 기록	사후 추적

SQL Agent도 위험한 실행 전에는 HITL 경계를 둘 수 있습니다

사용자 질문이 데이터베이스 실행으로 이어지므로 승인과 검토 루프가 중요합니다



HITL flow

Generate SQL

모델이 조회 쿼리를 제안합니다.

Review

사람 또는 정책이 위험 쿼리를 검토합니다.

Execute

허용된 쿼리만 실행합니다.

SQL 답변은 쿼리, 결과, 해석을 분리해 검증해야 합니다

최종 한국어 답변만 남기면 어떤 데이터에서 나온 결론인지 추적할 수 없습니다

- **남겨야 할 것**
  - 사용자 질문
  - 생성된 SQL
  - 실행 결과 행 수와 샘플
  - 최종 해석과 한계
- **거절해야 할 것**
  - 권한 밖 테이블 조회
  - 대량 개인정보 추출
  - 쓰기·삭제·스키마 변경

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 09

데이터 분석 에이전트

Deep Agents와 샌드박스 실행으로 데이터 분석 작업을 산출물 중심으로 수행합니다.

실습 · 05_advanced/07_data_analysis.ipynb

데이터 분석 실습은 산출물 파일과 해석 문장을 함께 검수합니다

분석 에이전트는 코드를 실행하므로 결과 재현성이 핵심입니다

체크포인트	확인할 내용	통과 기준
입력 데이터	파일과 스키마 확인	행 수와 컬럼이 기록됩니다
분석 코드	전처리·집계·시각화	재실행 가능합니다
산출물	표·차트·보고서 파일	작업 폴더에 남습니다
해석	결론과 한계	업무 판단과 연결됩니다

© 주식회사 똑똑한청년들 · 배움 에이아이

데이터 분석 리스크는 그럴듯한 차트가 검증 없이 제시될 때 커집니다

에이전트가 만든 분석도 사람이 만든 분석과 같은 검산 절차가 필요합니다

- **검산 항목**
  - 분모와 집계 단위
  - 결측과 이상치 처리
  - 필터와 기간 조건
- **보고 항목**
  - 분석 질문
  - 사용한 데이터
  - 결론과 한계

데이터 분석 에이전트는 질문·코드·파일·해석이 모두 남아야 합니다

분석 에이전트는 답변보다 재현 가능한 작업 흔적이 중요합니다

항목	핵심	실전 판단
질문	분석 목표와 지표 정의	모호하면 잘못된 코드가 생성됩니다
코드	전처리·집계·시각화 실행	재현성과 리뷰 대상입니다
파일	입력 데이터와 출력 산출물	작업 공간 경계가 필요합니다
해석	결과의 의미와 한계	업무 의사결정으로 연결합니다
검증	샘플 수, 결측, 단위 확인	그럴듯한 오답을 줄입니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Deep Agents는 파일 백엔드와 도구를 묶어 분석 작업 공간을 만듭니다

데이터 분석은 중간 파일과 코드 실행 결과가 남아야 하므로 파일 시스템 경계가 중요합니다

FilesystemBackend

분석 파일을 읽고 씁니다. 0.7 기준 FilesystemBackend의 기본 격리 설정과 root_dir 경계를 확인합니다.

tools

데이터 로딩과 코드 실행 도구를 제한합니다.

system_prompt

검증과 해석 기준을 명시합니다.

planning

분석 절차가 길어질 때는 TodoListMiddleware를 명시적으로 붙입니다.

07_data_analysis.ipynb

analysis agent

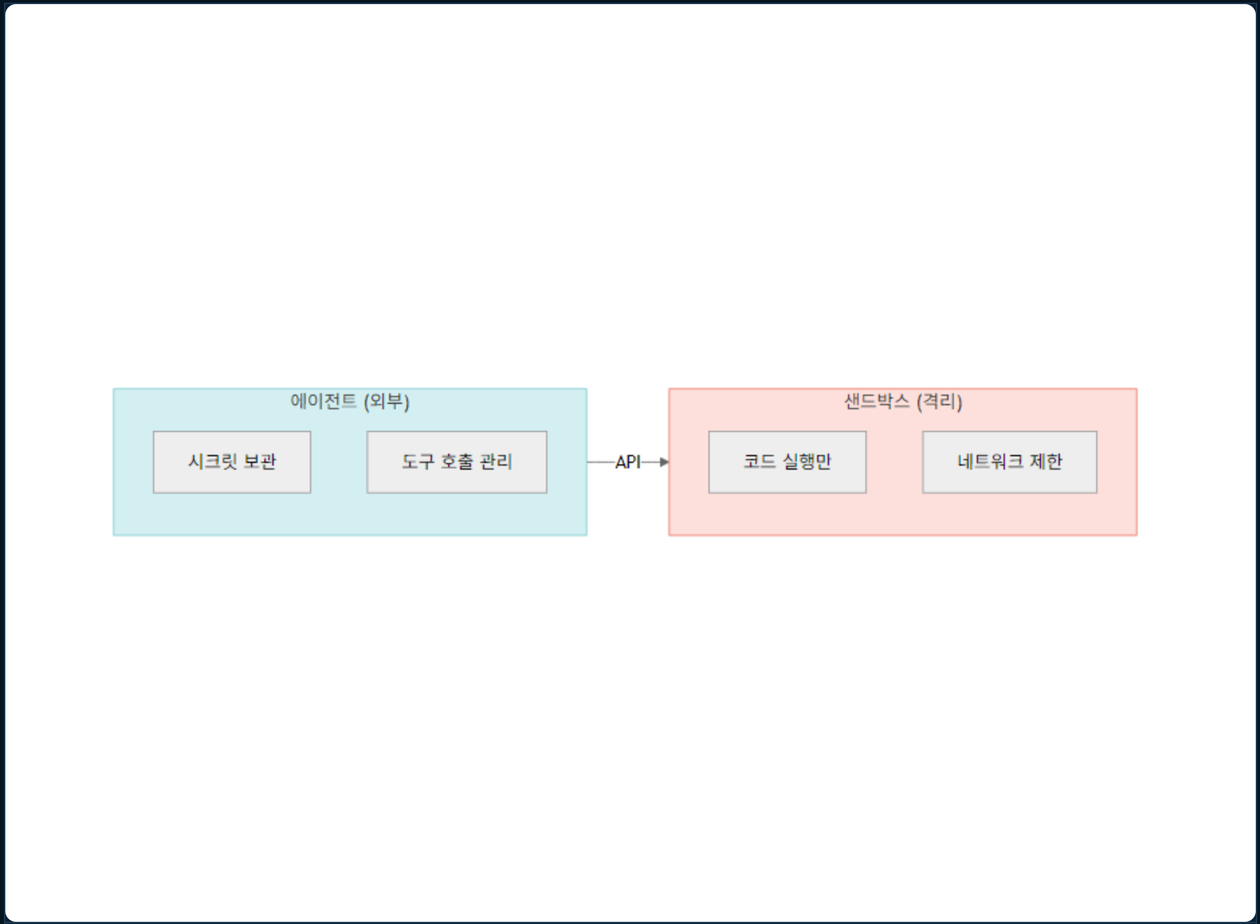
```python
from deepagents import create_deep_agent
from langchain.agents.middleware import TodoListMiddleware
from deepagents.backends import FilesystemBackend

analysis_agent = create_deep_agent(
    model=model,
    tools=[python_repl, read_file, write_file],
    middleware=[TodoListMiddleware()],
    backend=FilesystemBackend(root_dir="./analysis", virtual_mode=True),
    system_prompt="데이터 분석 과정과 한계를 함께 기록하세요.",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

분석 코드 실행은 에이전트 제어면과 샌드박스를 분리해서 봐야 합니다

데이터와 코드를 실행하는 곳에는 보안·비용·재현성 기준이 필요합니다



sandbox architecture

Agent

분석 계획과 해석을 담당합니다.

Sandbox

코드 실행과 파일 처리를 격리합니다.

Artifacts

표, 차트, 보고서 파일을 산출합니다.

Provider settings

OpenAI prompt_cache_options, Google thinking_config처럼 provider별 설정은 환경 회귀 항목으로 분리합니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

분석 결과는 숫자보다 검증 문장이 먼저 필요합니다

차트와 표가 있어도 단위, 필터, 결측 기준이 없으면 업무 의사결정에 쓰기 어렵습니다

검증 문장

- 사용한 데이터 기간과 필터를 밝힙니다
- 행 수, 결측, 이상치를 확인합니다
- 집계 단위와 분모를 설명합니다

해석 문장

- 결론과 근거 지표를 연결합니다
- 대체 설명과 한계를 적습니다
- 다음 분석 질문을 제안합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 10

Voice Agent

STT, Agent, TTS와 멀티모달 content blocks로 음성 에이전트 아키텍처를 설계합니다.

실습 · 05_advanced/08_voice_agent.ipynb

Voice Agent 실습은 정확도보다 지연·중단·승인 흐름을 함께 봐야 합니다

음성 UI에서는 사용자가 기다리는 시간과 끼어들기가 제품 품질을 좌우합니다

체크포인트	확인할 내용	통과 기준
STT 지연	음성 인식까지 걸리는 시간	사용자가 긴 침묵을 느끼지 않습니다
Agent 지연	도구 호출과 추론 시간	중간 상태를 안내합니다
TTS 제어	응답 재생과 중단	barge-in을 처리합니다
승인 흐름	위험 작업 확인	음성만으로 민감 작업을 실행하지 않습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Voice Agent 리스크는 오인식과 과잉 실행이 결합될 때 커집니다

잘못 들은 명령을 곧바로 도구 실행으로 연결하지 않아야 합니다

안전 장치

- 민감 작업은 다시 읽고 승인받습니다
- 사용자 발화를 텍스트로 확인할 수 있게 합니다
- 실패 시 텍스트 fallback을 제공합니다

로그 기준

- 음성 원문 저장 여부를 정책화합니다
- 인식 결과와 실행 명령을 분리해 기록합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Voice Agent는 STT, Agent, TTS가 지연 시간 안에서 이어지는 시스템입니다

음성은 정확도뿐 아니라 응답 지연과 중간 끊김이 사용자 경험을 좌우합니다

항목	핵심	실전 판단
STT	사용자 음성을 텍스트로 변환	실시간성, 화자 구분
Agent	텍스트 질문을 이해하고 도구를 사용	대화 상태와 권한
TTS	응답을 음성으로 변환	톤, 속도, 중단 처리
Streaming	부분 결과를 즉시 전달	지연 시간 체감 개선
Content blocks	텍스트·이미지·reasoning 구조화	멀티모달 응답 표준화

© 주식회사 똑똑한청년들 · 배움 에이아이

음성 에이전트 아키텍처는 세 가지 방식으로 비교합니다

어떤 컴포넌트가 실시간 스트림을 잡고 있는지에 따라 구현 난도가 달라집니다

Sandwich

STT → Agent → TTS를 순서대로 연결합니다. 가장 이해하기 쉽습니다.

`simple`

Realtime

모델 또는 런타임이 음성 스트림을 직접 다룹니다. 지연이 낮습니다.

`low latency`

Hybrid

중요 도구 호출은 텍스트 에이전트로, 대화 입출력은 음성으로 처리합니다.

`control`

Fallback

음성 실패 시 텍스트 UI로 전환합니다. 운영 안정성이 높습니다.

`resilience`

© 주식회사 똑똑한청년들 · 배움 에이아이

음성 에이전트는 끼어들기와 중단을 제품 요구사항으로 다뤄야 합니다

텍스트 채팅과 달리 사용자는 말하는 중간에 멈추거나 수정하거나 말을 끊을 수 있습니다

- **대화 제어**
  - 사용자 barge-in을 감지합니다
  - TTS 재생을 중단할 수 있어야 합니다
  - 부분 인식 결과와 최종 인식 결과를 구분합니다
- **운영 제어**
  - 민감 작업은 음성만으로 실행하지 않습니다
  - 승인 문구를 명확히 다시 읽어 줍니다
  - 오인식 로그와 재시도 경로를 남깁니다

© 주식회사 똑똑한청년들 · 배움 에이아이

output_version="v1"은 멀티모달 content blocks를 표준 형태로 받기 위한 설정입니다

음성·이미지·텍스트가 섞이는 앱에서는 응답 구조를 고정해야 프런트엔드 처리가 단순해집니다

TextContentBlock

일반 텍스트 응답입니다.

ImageContentBlock

이미지 입력 또는 출력과 연결됩니다.

Reasoning block

모델 제공자별 추론 블록을 표준 구조로 다룹니다.

08_voice_agent.ipynb

content blocks

```python
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
    output_version="v1",
)

response = model.invoke("음성 에이전트의 핵심 설계를 요약해줘")
print(response.content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 11

프로덕션

테스트, 평가, 관측성, 배포, 롤백을 에이전트 제품 출시 기준으로 묶습니다.

실습 · 05_advanced/09_production.ipynb

프로덕션 실습은 평가셋과 배포 체크리스트가 있어야 완료됩니다

기능이 돌아가도 평가와 롤백 기준이 없으면 출시 상태가 아닙니다

체크포인트	확인할 내용	통과 기준
평가셋	대표 질문과 기대 결과	릴리즈마다 재실행됩니다
관측성	trace와 비용 지표	실패 지점이 보입니다
보안	권한·PII·HITL 정책	감사 로그가 남습니다
롤백	프롬프트·도구·그래프 변경 이력	문제 발생 시 되돌릴 수 있습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

프로덕션 리스크는 모델 품질보다 변경 관리 부재에서 자주 발생합니다

프롬프트 한 줄 변경도 도구 호출 경로와 비용을 바꿀 수 있습니다

- **변경 전**
  - 회귀 평가셋을 실행합니다
  - 비용과 지연 변화를 비교합니다
  - 위험 도구 케이스를 재검증합니다
- **변경 후**
  - 실패 trace를 데이터셋에 추가합니다
  - 사용자 피드백을 평가 항목으로 바꿉니다

© 주식회사 똑똑한청년들 · 배움 에이아이

에이전트 테스트는 Unit, Integration, Evals로 나눠야 합니다

함수 테스트만 통과해도 실제 에이전트 행동은 실패할 수 있습니다

항목	핵심	실전 판단
Unit	도구 함수와 라우터 단위 검증	결정론적 테스트
Integration	모델·도구·그래프 연결 검증	mock 또는 fake model
Evals	실제 질문 세트 품질 평가	LangSmith dataset
Trajectory	도구 호출 경로 평가	예상 도구 순서와 비교
Regression	이전 실패 재발 방지	릴리즈 전 고정 평가

© 주식회사 똑똑한청년들 · 배움 에이아이

LangSmith 평가는 질문, 기대 답변, 실행 결과를 데이터셋으로 비교합니다

프로덕션에서는 한두 번의 수동 시연보다 반복 가능한 평가셋이 필요합니다

dataset

대표 질문과 기대 결과 모음입니다.

evaluator

정답성, 도구 경로, 안전성을 채점합니다.

experiment

모델·프롬프트 변경 전후를 비교합니다.

09_production.ipynb

LangSmith eval

```python
from langsmith import Client

client = Client()
# dataset = client.create_dataset("agent-regression")
# client.create_examples(dataset_id=dataset.id, examples=examples)

# evaluate(agent, data=dataset.name, evaluators=[trajectory_evaluator])
```

© 주식회사 똑똑한청년들 · 배움 에이아이

SmithDB 조회는 runs.query로 프로젝트·기간·projection을 명시합니다

runs.query는 projection을 명시하지 않으면 기본적으로 ID만 돌려줍니다. 분석에 필요한 필드는 uppercase selects로 요청합니다

project_ids

프로젝트 이름이 아니라 UUID 리스트로 조회 범위를 고정합니다.

time window

현재 시각 기준 window를 계산해 배포 전후 비교 구간을 재현합니다.

selects

기본 projection은 ID뿐입니다. 이름·시간·입출력이 필요하면 ID, NAME, START_TIME, END_TIME, INPUTS, OUTPUTS를 명시합니다.

09_production.ipynb — SmithDB runs.query

```
from datetime import datetime, timedelta, timezone
from langsmith import Client

client = Client()
project_id = "00000000-0000-0000-0000-000000000000"
now = datetime.now(timezone.utc)
selects = ["ID", "NAME", "START_TIME", "END_TIME", "INPUTS", "OUTPUTS"]

async for run in client.runs.query(
    project_ids=[project_id],
    min_start_time=now - timedelta(days=7),
    max_start_time=now,
    page_size=100,
    selects=selects,
):
    print(run["ID"], run.get("NAME"), run.get("START_TIME"))
```

© 주식회사 똑똑한청년들 · 배움 에이아이

LangSmith migration은 runs, feedback, sharing 영역을 따로 판단합니다

모든 legacy 호출을 한 번에 치환하지 않고 서비스 지원 범위와 예외 경로를 확인합니다

영역	확인할 것	덱 판단
runs.query	project_ids, min/max_start_time, page_size, uppercase selects	selects 생략 시 기본 ID projection만 반환
feedback	점수·코멘트·run 연결 API	평가 루프와 함께 검증
sharing	공유 링크와 self-hosted 지원 차이	배포 환경별 조건부 항목
exceptions	클라우드와 self-hosted의 오류 경로	전역 치환 금지

© 주식회사 똑똑한청년들 · 배움 에이아이

evaluation tracing은 비용과 디버깅 사이에서 선택합니다

evaluate 호출은 유지하되 evaluator 내부 trace를 남길지 별도 옵션으로 결정합니다

옵션	효과	운영 판단
evaluate 기본	실험 결과와 run 비교를 남김	회귀 평가의 기준 경로
disable_evaluator_tracing	evaluator 내부 trace 비용을 줄임	채점 과정 디버깅이 덜 필요할 때
grader hooks	criterion별 실패 원인 기록	루브릭 coverage와 함께 검수
피드백 연결	사용자·자동 채점 결과를 run에 연결	운영 모니터링으로 이어짐

© 주식회사 똑똑한청년들 · 배움 에이아이

공식 문서 이동과 신규 항목은 기존 트랙에 연결해 읽습니다

Code, dynamic subagents, fault tolerance, multimodal, retrieval은 새 파일을 늘리기보다 기존 주제의 기준점을 갱신합니다

항목	연결 위치	수업 판단
Code / plugins	Deep Agents code 문서와 통합 트랙	코어 노트북에 CLI 의존성 주입 금지
dynamic subagents	서브에이전트 설계와 fork 비교	정적 선언과 동적 생성을 분리
fault tolerance	LangGraph durable execution과 production	재시도·중단·복구를 한 루프로 설명
multimodal / retrieval	RAG·voice·data analysis	기존 실습과 중복을 확인한 뒤 확장

© 주식회사 똑똑한청년들 · 배움 에이아이

운영 모니터링은 답변 품질과 시스템 비용을 함께 봐야 합니다

에이전트는 모델 호출, 도구 호출, 사람 승인, 외부 API 비용이 함께 발생합니다

항목	핵심	실전 판단
품질	정답성, 근거성, 거절 정확도	평가셋과 사용자 피드백
경로	도구 호출 순서와 실패 지점	trace와 trajectory
비용	토큰, 도구 API, 실행 시간	예산과 호출 제한
지연	최종 응답과 중간 스트림 시간	사용자 경험
안전	HITL, PII, 권한 위반	감사 로그와 알림

© 주식회사 똑똑한청년들 · 배움 에이아이

출시 기준은 기능 완료가 아니라 실패 대응 절차가 준비됐는지로 판단합니다

고급 에이전트는 실패하지 않는 시스템이 아니라 실패를 발견하고 되돌릴 수 있는 시스템이어야 합니다

- **출시 전**
  - 회귀 평가셋을 통과합니다
  - 민감 도구의 승인 정책을 검증합니다
  - 비용과 지연 예산을 정합니다
- **출시 후**
  - 실패 trace를 수집합니다
  - 오답을 데이터셋에 추가합니다
  - 프롬프트, 도구, 그래프 변경을 실험 단위로 비교합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

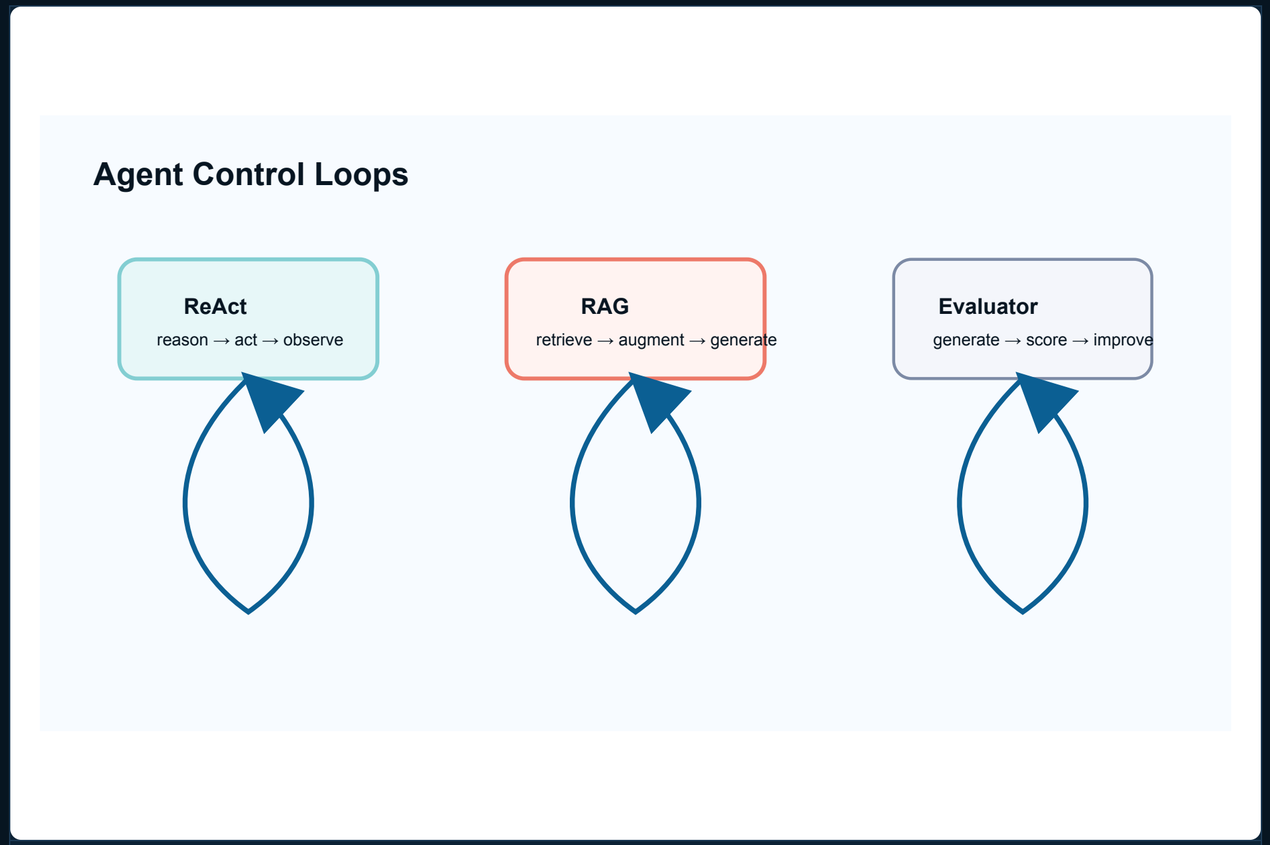
REFERENCE

개념 근거와 고급 패턴

고급 패턴 설명은 ReAct, RAG, Pregel 논문, Anthropic 에이전트 패턴 문서, LangChain/LangGraph 공식 문서를 기준으로 보강합니다.

고급 에이전트 패턴은 control loop, retrieval loop, evaluation loop로 나눠 설명할 수 있습니다

ReAct는 추론과 행동의 교차 루프를, RAG는 검색과 생성 결합을, Anthropic은 workflow와 agent 패턴의 구분을 제시합니다.



agent loops from trusted sources

ReAct

reasoning과 acting을 번갈아 수행합니다.

RAG

parametric memory와 non-parametric memory를 결합합니다.

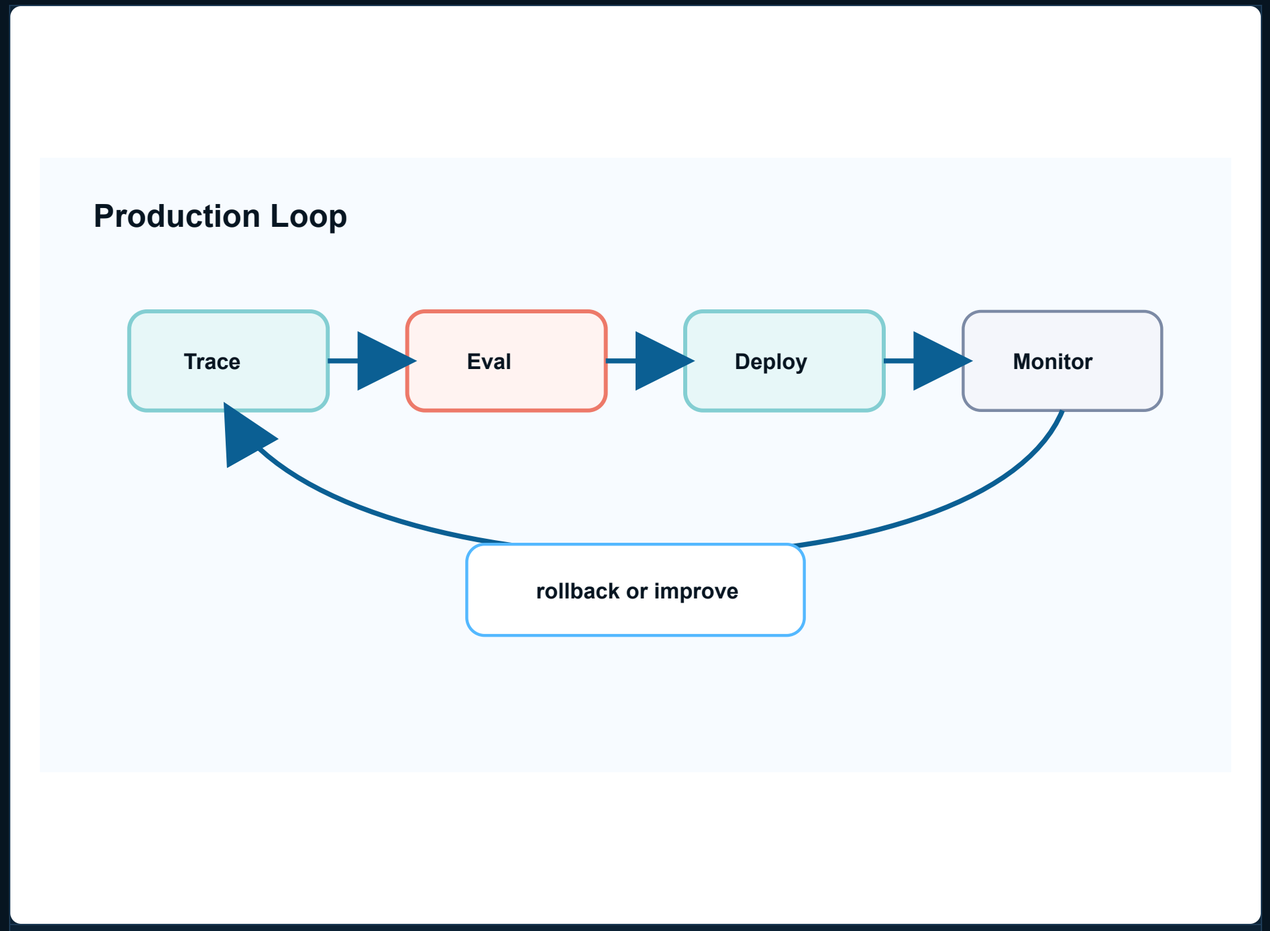
Evaluator

생성 결과를 평가하고 개선하는 루프입니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

프로덕션 에이전트는 trace, eval, guardrail, rollback을 한 루프로 관리합니다

공식 LangSmith/LangGraph 문서와 Anthropic 패턴은 평가와 관찰 가능성을 운영 루프의 핵심으로 봅니다.



production evaluation loop

Trace

도구 호출과 상태 변화를 기록합니다.

Eval

대표 질문 세트로 회귀를 확인합니다.

Rollback

문제 변경을 되돌릴 수 있어야 합니다.

고급 개념 보강 출처는 논문과 공식 문서를 함께 사용합니다

응용별 개념은 가장 직접적인 신뢰 출처를 기준으로 설명합니다.

개념	출처	덱 반영
Tool-using agent	ReAct, arXiv:2210.03629	reason-act-observe 루프와 trajectory
RAG	Lewis et al., NeurIPS 2020	parametric/non-parametric memory와 provenance
Workflow patterns	Anthropic Building Effective Agents	prompt chaining, routing, parallelization, orchestrator-worker, evaluator-optimizer
Durable state	LangGraph official docs	persistence, HITL, time travel, durable execution

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

00_migration.ipynb

00_migration.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · `05_advanced/00_migration.ipynb`

00_migration 코드 01

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

00_migration.ipynb

cell 5

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain.agents import create_agent

model = ChatOpenAI(model="gpt-5.4")

@tool
def add(a: int, b: int) → int:
    """두 수를 더합니다."""
    return a + b

# v0: from langgraph.prebuilt import create_react_agent
# v1: from langchain.agents import create_agent
agent = create_agent(
    model=model,
    tools=[add],
    system_prompt="당신은 수학 어시스턴트입니다.",  # v0: prompt=
)
print("\u2713 v1 에이전트 생성 완료")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

00_migration 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

00_migration.ipynb

cell 6

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON \u2014 project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

00_migration 코드 03

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

00_migration.ipynb

cell 9

```
from langchain.agents import AgentState, create_agent

class CustomState(AgentState):
    user_id: str

agent = create_agent(
    model=model,
    tools=[add],
    state_schema=CustomState,
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "3+4는 얼마인가요?"}],
    "user_id": "user_123",
}, config=lf_config)
print(result["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

00_migration 코드 04 (1/2)

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

00_migration.ipynb

cell 11

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime

@dataclass
class UserContext:
    user_id: str
    role: str

@tool
def get_role(runtime: ToolRuntime[UserContext]) -> str:
    """현재 사용자의 역할을 조회합니다."""
    return f"사용자 {runtime.context.user_id}의 역할: {runtime.context.role}"

agent = create_agent(
    model=model,
    tools=[get_role],
    context_schema=UserContext,
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "제 역할이 뭔가요?"}]},
    context=UserContext(user_id="u1", role="admin"),
```

00_migration 코드 04 (2/2)

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

00_migration.ipynb

cell 11

```
    config=lf_config,
)
print(result["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

00_migration 코드 05

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

00_migration.ipynb

cell 13

```
from langchain.agents.middleware import dynamic_prompt, ModelRequest

@dynamic_prompt
def role_based_prompt(request: ModelRequest) -> str:
    role = request.runtime.context.role
    if role == "admin":
        return "당신은 전체 접근 권한을 가진 관리자 어시스턴트입니다."
    return "당신은 읽기 전용 어시스턴트입니다."

agent = create_agent(
    model=model,
    tools=[add],
    context_schema=UserContext,
    middleware=[role_based_prompt],
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "안녕하세요!"}]},
    context=UserContext(user_id="u1", role="admin"),
    config=lf_config,
)
print(result["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

00_migration 코드 06

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

00_migration.ipynb

cell 15

```
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage

@wrap_tool_call
def handle_errors(request, handler):
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"도구 오류: {e}",
            tool_call_id=request.tool_call["id"],
        )

agent = create_agent(
    model=model,
    tools=[add],
    middleware=[handle_errors],
)
print("\u2713 에러 핸들링 미들웨어 적용")
```

00_migration 코드 07

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

00_migration.ipynb

cell 18

```
# 표준 콘텐츠 블록

response = model.invoke(
    "인사해 주세요.",
    config=lf_config
)

for block in response.content_blocks:
    print(f"  [{block['type']}] {block.get('text', '')[:100]}")

# .text 프로퍼티 (v0: .text() 메서드 → v1: .text 프로퍼티)

print(f"\n.text: {response.text[:100]}")
```

00_migration 코드 08

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

00_migration.ipynb

cell 20

```python
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "5+3은 얼마인가요?"}]},
    context=UserContext(user_id="u1", role="admin"),
    stream_mode="updates",
    config=lf_config,
):
    for node_name, update in chunk.items():
        # v1: node_name은 "model" (v0에서는 "agent")
        if "messages" in update:
            last = update["messages"][-1]
            if hasattr(last, "content") and last.content:
                print(f"[{node_name}] {last.content[:200]}")
```

CODE

01_middleware.ipynb

01_middleware.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 05_advanced/01_middleware.ipynb

01_middleware 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

01_middleware.ipynb cell 3

```python
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

01_middleware.ipynb

cell 4

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON — project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

01_middleware 코드 03

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

01_middleware.ipynb

cell 6

```python
from langchain.agents import create_agent
from langchain.agents.middleware import (
    SummarizationMiddleware,
    HumanInTheLoopMiddleware,
)

agent = create_agent(
    model="gpt-5.4", tools=[],
    middleware=[
        SummarizationMiddleware(model="gpt-5.4-mini", trigger=("messages", 50)),
        HumanInTheLoopMiddleware(interrupt_on={}),
    ],
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 04

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 8

```python
from langchain.agents.middleware import SummarizationMiddleware

summarizer = SummarizationMiddleware(
    model="gpt-5.4-mini",
    trigger=("tokens", 4000),
    keep=("messages", 20),
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 05

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 10

```
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "send_email": {"allowed_decisions": ["approve", "edit", "reject"]},
        "read_email": False,
    }
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 06

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

01_middleware.ipynb

cell 11

```
agent = create_agent(
    model="gpt-5.4", tools=[],
    checkpointer=InMemorySaver(),
    middleware=[hitl],
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 07

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

01_middleware.ipynb

cell 12

```
from langgraph.types import Command

# result = agent.invoke(Command(resume="approve"), config=lf_config)
# result = agent.invoke(Command(resume={"type": "edit", "args": {"to": "new@ex.com"}}), config=lf_config)
# result = agent.invoke(Command(resume={"type": "reject", "reason": "필요 없음"}), config=lf_config)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 08

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 14

```python
from langchain.agents.middleware import ModelCallLimitMiddleware

model_limit = ModelCallLimitMiddleware(
    thread_limit=10,
    run_limit=5,
    exit_behavior="end",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 09

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 15

```python
from langchain.agents.middleware import ToolCallLimitMiddleware

# 전역 제한
global_tool_limit = ToolCallLimitMiddleware(thread_limit=20, run_limit=10)

# 특정 도구 제한
search_limit = ToolCallLimitMiddleware(
    tool_name="search",
    thread_limit=5, run_limit=3,
    exit_behavior="continue",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 10

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 17

```python
from langchain.agents.middleware import ModelFallbackMiddleware

# gpt-5.4 실패 → gpt-5.4-mini → claude
fallback = ModelFallbackMiddleware(
    "gpt-5.4-mini",
    "claude-sonnet-4-6",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 11

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 19

```
from langchain.agents.middleware import PIIMiddleware

email_pii = PIIMiddleware("email", strategy="redact", apply_to_input=True)
card_pii = PIIMiddleware("credit_card", strategy="mask", apply_to_input=True)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 12

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 20

```
# 커스텀 탐지기: 정규식 문자열
api_key_pii = PIIMiddleware(
    "api_key",
    detector=r"sk-[a-zA-Z0-9]{32}",
    strategy="block",
)
```

01_middleware 코드 13

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 21

```
import re

# 커스텀 탐지기: 컴파일된 정규식
phone_pii = PIIMiddleware(
    "phone_number",
    detector=re.compile(r"\+?\d{1,3}[\s.-]?\d{3,4}[\s.-]?\d{4}"),
    strategy="mask",
)
```

01_middleware 코드 14

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 22

```
# 커스텀 탐지기: 함수 (SSN 예시)
def detect_ssn(content: str) → list[dict]:
    matches = []
    for m in re.finditer(r"\d{3}-\d{2}-\d{4}", content):
        first = int(m.group(0)[:3])
        if first not in [0, 666] and not (900 ⩽ first ⩽ 999):
            matches.append({"text": m.group(0), "start": m.start(), "end": m.end()})
    return matches

ssn_pii = PIIMiddleware("ssn", detector=detect_ssn, strategy="hash")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 15

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 24

```python
from langchain.agents.middleware import LLMToolSelectorMiddleware

tool_selector = LLMToolSelectorMiddleware(
    model="gpt-5.4-mini",
    max_tools=3,
    always_include=["search"],
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 16

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

01_middleware.ipynb

cell 26

```python
# AnthropicPromptCachingMiddleware — 긴 시스템 프롬프트/도구를 캐시 히트로
from langchain_anthropic import ChatAnthropic
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware
from langchain.agents import create_agent

cached_agent = create_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    system_prompt="<긴 시스템 프롬프트>",
    middleware=[AnthropicPromptCachingMiddleware(ttl="5m", min_messages_to_cache=0)],
)
```

01_middleware 코드 17

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

01_middleware.ipynb

cell 27

```python
# BedrockPromptCachingMiddleware — Bedrock 호스팅 Claude 동일 캐싱
from langchain_aws import ChatBedrockConverse
from langchain_aws.middleware.prompt_caching import BedrockPromptCachingMiddleware

bedrock_agent = create_agent(
    model=ChatBedrockConverse(model="us.anthropic.claude-sonnet-4-5-20250929-v1:0"),
    system_prompt="<긴 시스템 프롬프트>",
    middleware=[BedrockPromptCachingMiddleware(ttl="1h")],
)
```

01_middleware 코드 18

이 코드는 Deep Agent 생성과 하네스 옵션 연결을 보여줍니다.

01_middleware.ipynb

cell 28

```python
# PatchToolCallsMiddleware — 잘못된 도구 호출을 도구 노드 도달 전에 보정
from deepagents.middleware.patch_tool_calls import PatchToolCallsMiddleware

patched_agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[],
    middleware=[PatchToolCallsMiddleware()],
)
# 참고: create_deep_agent(...) 사용 시 기본 미들웨어 스택에 자동 포함
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 19

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

01_middleware.ipynb

cell 29

```python
# ContextEditingMiddleware — 토큰 임계 도달 시 오래된 도구 출력을 placeholder로 교체
from langchain.agents.middleware import ContextEditingMiddleware, ClearToolUsesEdit

ctx_edit_agent = create_agent(
    model="gpt-5.4",
    tools=[],
    middleware=[
        ContextEditingMiddleware(
            edits=[ClearToolUsesEdit(trigger=100_000, keep=3)],
        ),
    ],
)
```

01_middleware 코드 20

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

01_middleware.ipynb

cell 31

```python
from langchain.agents.middleware import before_model

@before_model
def log_before(state, runtime):
    """모델 호출 전 메시지 수를 기록합니다."""
    print(f"[LOG] 메시지 {len(state.get('messages', []))}개")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 21

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

01_middleware.ipynb

cell 32

```python
from langchain.agents.middleware import after_model

@after_model
def validate_output(state, runtime):
    """가드: 금지된 콘텐츠를 차단합니다."""
    last = state["messages"][-1].content
    if "FORBIDDEN" in last:
        return {"jump_to": "end"}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 22

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 33

```python
from typing import Callable
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse

@wrap_model_call
def retry_on_error(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """실패 시 모델 호출을 최대 2회 재시도합니다."""
    for attempt in range(3):
        try:
            return handler(request)
        except Exception:
            if attempt == 2: raise
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 23

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb cell 34

```python
# wrap_tool_call — 도구 호출을 감싸 감사 로그/에러 변환
from typing import Callable
from langchain.agents.middleware import wrap_tool_call, ToolCallRequest
from langchain.messages import ToolMessage
from langgraph.types import Command

@wrap_tool_call
def audit_tool(
    request: ToolCallRequest,
    handler: Callable[[ToolCallRequest], ToolMessage | Command],
) -> ToolMessage | Command:
    """도구 호출 전후로 감사 로그를 남기고 예외를 ToolMessage 로 변환."""
    name = request.tool_call["name"]
    print(f"[AUDIT] tool={name} args={request.tool_call['args']}")
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"도구 오류: {e}",
            tool_call_id=request.tool_call["id"],
        )
```

01_middleware 코드 24

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 35

```python
# request.override(...) — wrap_model_call 에서 모델/도구/시스템 프롬프트를 단일 호출 한정으로 교체
@wrap_model_call
def dynamic_config(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) → ModelResponse:
    """상태 기반으로 모델 호출 시 system_prompt 와 tools 를 동적으로 교체."""
    if request.state.get("expert_mode"):
        request = request.override(
            system_prompt="당신은 시니어 분석가입니다.",
            # tools=[...specialist_tools],
        )
    return handler(request)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 25

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

01_middleware.ipynb

cell 36

```python
# @hook_config(can_jump_to=[...]) — after_model 에서 합법 점프 대상 선언
from typing import Any
from langchain.agents.middleware import after_model, hook_config, AgentState
from langchain.messages import AIMessage
from langgraph.runtime import Runtime

@after_model
@hook_config(can_jump_to=["end"])
def block_forbidden(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    last = state["messages"][-1]
    if "BLOCKED" in getattr(last, "content", ""):
        return {
            "messages": [AIMessage("요청에 응답할 수 없습니다.")],
            "jump_to": "end",
        }
    return None
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 26

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 37

```python
# ExtendedModelResponse — wrap_model_call 에서 모델 응답 + 영구 상태 업데이트를 함께 반환
from langchain.agents.middleware import ExtendedModelResponse

@wrap_model_call
def track_tokens(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ExtendedModelResponse:
    response = handler(request)
    return ExtendedModelResponse(
        model_response=response,
        command=Command(update={"last_model_call_tokens": 150}),
    )
```

01_middleware 코드 27

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

01_middleware.ipynb

cell 38

```python
from langchain.agents.middleware import AgentMiddleware

class AuditMiddleware(AgentMiddleware):
    def __init__(self, log_file="audit.log"):
        self.log_file = log_file
    def before_model(self, state, config):
        print(f"[AUDIT] before → {self.log_file}")
    def after_model(self, state, config):
        print(f"[AUDIT] after → {self.log_file}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 28

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

01_middleware.ipynb

cell 40

```python
@before_model
def mw_a(state, runtime): print("before A")

@before_model
def mw_b(state, runtime): print("before B")

@before_model
def mw_c(state, runtime): print("before C")

# 실행: A → B → C (after_model이면 C → B → A)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 29

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

01_middleware.ipynb

cell 42

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, ModelFallbackMiddleware,
    ModelCallLimitMiddleware, SummarizationMiddleware,
    HumanInTheLoopMiddleware, LLMToolSelectorMiddleware,
    ContextEditingMiddleware, ClearToolUsesEdit,
)
from deepagents.middleware.patch_tool_calls import PatchToolCallsMiddleware
from langgraph.checkpoint.memory import InMemorySaver
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_middleware 코드 30

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

01_middleware.ipynb

cell 43

```
middleware_stack = [
    PIIMiddleware("email", strategy="redact", apply_to_input=True),
    PatchToolCallsMiddleware(),
    ModelFallbackMiddleware("gpt-5.4-mini", "claude-sonnet-4-6"),
    ModelCallLimitMiddleware(thread_limit=50, run_limit=10),
    ContextEditingMiddleware(edits=[ClearToolUsesEdit(trigger=100_000, keep=3)]),
    SummarizationMiddleware(model="gpt-5.4-mini", trigger=("fraction", 0.8)),
]

production_agent = create_agent(
    model="gpt-5.4", tools=[], checkpointer=InMemorySaver(), middleware=middleware_stack,
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

02_multi_agent_subagents.ipynb

02_multi_agent_subagents.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 05_advanced/02_multi_agent_subagents.ipynb

02_multi_agent_subagents 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

02_multi_agent_subagents.ipynb cell 3

```python
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

02_multi_agent_subagents.ipynb

cell 4

```python
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON — project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

02_multi_agent_subagents 코드 03

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

02_multi_agent_subagents.ipynb cell 7

```python
from langchain_core.tools import tool

@tool
def create_calendar_event(
    title: str, start_time: str, end_time: str,
    attendees: list[str] = None,
) -> str:
    """새 캘린더 이벤트를 생성합니다."""
    return f"이벤트 '{title}' 생성됨: {start_time} ~ {end_time}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 04

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

02_multi_agent_subagents.ipynb

cell 8

```
@tool
def read_calendar_events(date: str) → str:
    """날짜(YYYY-MM-DD)의 캘린더 이벤트를 조회합니다."""
    return f"{date}에 이벤트가 없습니다."
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 05

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

02_multi_agent_subagents.ipynb

cell 9

```
@tool
def send_email(to: str, subject: str, body: str) → str:
    """이메일 메시지를 전송합니다."""
    return f"{to}에게 이메일 전송됨: '{subject}'"

@tool
def read_emails(folder: str = "inbox", limit: int = 10) → str:
    """폴더에서 최근 이메일을 읽습니다."""
    return f"{folder}에 이메일 3개"
```

02_multi_agent_subagents 코드 06

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

02_multi_agent_subagents.ipynb

cell 10

```
@tool
def search_emails(query: str, limit: int = 10) -> str:
    """검색어로 이메일을 검색합니다."""
    return f"'{query}' 검색 결과 2건"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 07

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 12

```
from langchain.agents import create_agent

calendar_agent = create_agent(
    model="gpt-5.4",
    tools=[create_calendar_event, read_calendar_events],
    system_prompt="당신은 캘린더 어시스턴트입니다. ISO 8601 날짜 형식을 사용하세요.",
    name="calendar_agent",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 08

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 13

```
email_agent = create_agent(
    model="gpt-5.4",
    tools=[send_email, read_emails, search_emails],
    system_prompt="당신은 이메일 어시스턴트입니다. 메시지를 전문적으로 작성하세요.",
    name="email_agent",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 09

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

02_multi_agent_subagents.ipynb

cell 15

```
@tool("schedule_event", description="캘린더 이벤트 예약")
def call_calendar(query: str) → str:
    """캘린더 작업을 캘린더 에이전트에 위임합니다."""
    result = calendar_agent.invoke(
        {"messages": [{"role": "user", "content": query}]},
        config=lf_config,
)
    return result["messages"][-1].content
```

02_multi_agent_subagents 코드 10

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

02_multi_agent_subagents.ipynb

cell 16

```
@tool("manage_email", description="이메일 전송, 읽기, 검색")
def call_email(query: str) → str:
    """이메일 작업을 이메일 에이전트에 위임합니다."""
    result = email_agent.invoke(
        {"messages": [{"role": "user", "content": query}]},
        config=lf_config,
)
    return result["messages"][-1].content
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 11

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 18

```
supervisor = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt=(
        "당신은 개인 비서입니다. 복잡한 요청을 "
        "하위 작업으로 분해하고 적절한 에이전트에 위임하세요."
    ),
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 12

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

02_multi_agent_subagents.ipynb

cell 20

```
# response = supervisor.invoke({
#     "messages": [{"role": "user",
#         "content": "내일 2시에 Sarah와 미팅 잡고 "
#                    "초대 이메일 보내줘."}]
# }, config=lf_config)
# print(response["messages"][-1].content)
```

02_multi_agent_subagents 코드 13

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 22

```
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

hitl = HumanInTheLoopMiddleware(interrupt_on={
    "schedule_event": {"allowed_decisions": ["approve", "edit", "reject"]},
    "manage_email": {"allowed_decisions": ["approve", "reject"]},
})
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 14

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 23

```
supervisor_hitl = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    checkpointer=InMemorySaver(),
    middleware=[hitl],
    system_prompt="당신은 개인 비서입니다.",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 15

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 24

```
from langgraph.types import Command

# result = supervisor_hitl.invoke(Command(resume="approve"), config=lf_config)
# result = supervisor_hitl.invoke(Command(resume={"type": "reject", "reason": "마음이 바뀌었습니다"}), config=lf_config)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 16

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 26

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import AgentState, create_agent

@dataclass
class UserContext:
    user_email: str
    user_name: str
    timezone: str

class SupervisorState(AgentState):
    # 부모 에이전트가 보유한 추가 상태 키 예시
    user_id: str
```

02_multi_agent_subagents 코드 17

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 27

```
supervisor_ctx = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt="당신은 개인 비서입니다.",
    context_schema=UserContext,
    state_schema=SupervisorState,
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 18

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

02_multi_agent_subagents.ipynb

cell 28

```
# 도구에서 부모 상태 + 컨텍스트 동시 접근
@tool
def send_email_ctx(
    to: str, subject: str, body: str,
    runtime: ToolRuntime[UserContext, SupervisorState],
) → str:
    """현재 사용자로 이메일을 전송합니다."""
    sender = runtime.context.user_email
    user_id = runtime.state.get("user_id", "anon")
    return f"[{user_id}] {sender}에서 {to}로 이메일 전송: '{subject}'"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 19 (1/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

02_multi_agent_subagents.ipynb cell 30

```
import uuid
from datetime import datetime

job_store: dict[str, dict] = {}

@tool("start_async_task", description="서브에이전트 백그라운드 기동, task id 반환")
def start_async_task(agent_name: str, instruction: str) → str:
    """비동기 서브에이전트 작업을 시작하고 task id를 반환합니다."""
    task_id = "task_" + uuid.uuid4().hex[:12]
    job_store[task_id] = {
        "agent": agent_name,
        "status": "running",
        "instruction": instruction,
        "result": None,
        "created_at": datetime.utcnow().isoformat(),
    }
    return f"started {task_id} (agent={agent_name})"

@tool("check_async_task", description="task 상태 조회 / 완료 시 결과 추출")
def check_async_task(task_id: str) → str:
    job = job_store.get(task_id)
    if not job:
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 19 (2/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

02_multi_agent_subagents.ipynb

cell 30

```
    return f"not_found: {task_id}"
# 학습용 스텁: 한 번 호출되면 완료로 표시
if job["status"] == "running":
    job["status"] = "success"
    job["result"] = f"{job['agent']} 결과: {job['instruction'][:60]}"
return f"{task_id} status={job['status']} result={job.get('result')!r}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 20 (1/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

02_multi_agent_subagents.ipynb cell 31

```
@tool("update_async_task", description="실행 중 task 에 follow-up 지시 주입")
def update_async_task(task_id: str, instruction: str) → str:
    job = job_store.get(task_id)
    if not job:
        return f"not_found: {task_id}"
    job["instruction"] = instruction
    job["status"] = "running"   # interrupt 기반 재기동
    return f"updated {task_id} with new instruction"

@tool("cancel_async_task", description="task 취소")
def cancel_async_task(task_id: str) → str:
    job = job_store.get(task_id)
    if not job:
        return f"not_found: {task_id}"
    job["status"] = "cancelled"
    return f"cancelled {task_id}"

@tool("list_async_tasks", description="추적 중 task 일괄 조회")
def list_async_tasks() → str:
    if not job_store:
        return "no tasks"
    return "\n".join(
```

02_multi_agent_subagents 코드 20 (2/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 31

```
        f"{tid} agent={j['agent']} status={j['status']}"
        for tid, j in job_store.items()
    )
```

02_multi_agent_subagents 코드 21

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

02_multi_agent_subagents.ipynb

cell 33

```
from typing import Literal

AGENT_REGISTRY = {"calendar": calendar_agent, "email": email_agent}

@tool("delegate", description="전문 에이전트에 위임")
def dispatch(agent_name: Literal["calendar", "email"], query: str) → str:
    """지정된 서브에이전트에 작업을 라우팅합니다."""
    agent = AGENT_REGISTRY[agent_name]
    result = agent.invoke({"messages": [{"role": "user", "content": query}]}, config=lf_config)
    return result["messages"][-1].content
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_multi_agent_subagents 코드 22

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

02_multi_agent_subagents.ipynb cell 34

```
supervisor_dispatch = create_agent(
    model="gpt-5.4",
    tools=[dispatch],
    system_prompt=(
        "delegate 도구를 사용하여 작업을 라우팅하세요. "
        "에이전트: 'calendar'(일정), 'email'(이메일)."
    ),
)
```

02_multi_agent_subagents 코드 23 (1/2)

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 35

```
# Deep Agents — SubAgentMiddleware 로 일괄 등록 (수동 @tool 래핑 불필요)
from deepagents.middleware.subagents import SubAgentMiddleware

supervisor_dag = create_agent(
    model="claude-sonnet-4-6",
    middleware=[
        SubAgentMiddleware(
            backend=StateBackend(),
            subagents=[
                {
                    "name": "calendar",
                    "description": "캘린더 이벤트 생성/조회",
                    "system_prompt": "당신은 캘린더 어시스턴트입니다. ISO 8601 사용.",
                    "tools": [create_calendar_event, read_calendar_events],
                    "model": "claude-sonnet-4-6",
                },
                {
                    "name": "email",
                    "description": "이메일 전송/읽기/검색",
                    "system_prompt": "당신은 이메일 어시스턴트입니다.",
                    "tools": [send_email, read_emails, search_emails],
                    "model": "gpt-5.4",   # 서브에이전트별 모델 오버라이드
```

02_multi_agent_subagents 코드 23 (2/2)

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

02_multi_agent_subagents.ipynb

cell 35

```
                },
            ],
        ),
    ],
)
# 감독자는 자동 주입된 `task(subagent_name, instruction)` 도구로 위임
```

CODE

03_multi_agent_handoffs_router.ipyn

03_multi_agent_handoffs_router.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 05_advanced/03_multi_agent_handoffs_router.ipynb

03_multi_agent_handoffs_router 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

03_multi_agent_handoffs_router.ipynb cell 4

```python
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

03_multi_agent_handoffs_router.ipynb cell 5

```python
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON — project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

03_multi_agent_handoffs_router 코드 03

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

03_multi_agent_handoffs_router.ipynb cell 8

```python
from langchain.agents import AgentState
from typing import Literal

class SupportState(AgentState):
    current_step: Literal[
        "identify_customer", "diagnose_issue",
        "resolve_issue", "close_ticket",
    ] = "identify_customer"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 04

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 10

```python
from langchain_core.tools import tool
from langgraph.types import Command

# --- Identify Customer ---
@tool
def lookup_customer(email: str) -> Command:
    """이메일로 고객을 조회합니다."""
    return Command(
        update={"customer": {"name": "Alice", "id": "C-1234"}, "current_step": "diagnose_issue"},
        result="고객 찾음: Alice (C-1234). 진단 단계로 이동합니다.",
    )
```

03_multi_agent_handoffs_router 코드 05

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

03_multi_agent_handoffs_router.ipynb

cell 11

```
# --- Diagnose Issue ---
@tool
def check_service_status(service_name: str) -> str:
    """서비스의 현재 상태를 확인합니다."""
    return f"서비스 '{service_name}': 정상 (99.9% 가동률)"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 06

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 12

```
@tool
def escalate_to_resolve(diagnosis: str) → Command:
    """진단 후 해결 단계로 이동합니다."""
    return Command(
        update={"diagnosis": diagnosis, "current_step": "resolve_issue"},
        result=f"진단 완료: {diagnosis}. 해결 단계로 이동합니다.",
    )
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 07

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

03_multi_agent_handoffs_router.ipynb cell 13

```python
# --- Resolve Issue ---
@tool
def apply_fix(fix_type: str, customer_id: str) -> Command:
    """고객 계정에 수정 사항을 적용합니다."""
    return Command(
        update={"resolution": {"type": fix_type}},
        result=f"수정 적용됨: {customer_id}에 {fix_type}",
    )
```

03_multi_agent_handoffs_router 코드 08

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 14

```
@tool
def mark_resolved(summary: str) -> Command:
    """이슈를 해결 완료로 표시하고 종료 단계로 이동합니다."""
    return Command(
        update={"current_step": "close_ticket", "resolution_summary": summary},
        result="해결됨. 종료 단계로 이동합니다.",
    )
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 09

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 15

```
# 정식 핸드오프 시그니처 — ToolRuntime.tool_call_id 를 ToolMessage 에 echo
from langchain.tools import ToolRuntime
from langchain.messages import ToolMessage

@tool
def transfer_to_resolve(
    diagnosis: str,
    runtime: ToolRuntime[None, SupportState],
) → Command:
    """진단 후 해결 단계로 이동 (tool_call_id echo 패턴)."""
    return Command(update={
        "current_step": "resolve_issue",
        "diagnosis": diagnosis,
        "messages": [ToolMessage(
            content=f"진단 완료: {diagnosis}. 해결 단계로 이동합니다.",
            tool_call_id=runtime.tool_call_id,
        )],
    })
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 10

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

03_multi_agent_handoffs_router.ipynb

cell 16

```
# --- Close Ticket ---
@tool
def send_satisfaction_survey(customer_id: str) -> str:
    """만족도 설문을 전송합니다."""
    return "설문 전송 완료."

@tool
def close_ticket(ticket_id: str, notes: str) -> str:
    """지원 티켓을 종료합니다."""
    return f"티켓 {ticket_id} 종료됨."
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 11

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

03_multi_agent_handoffs_router.ipynb

cell 18

```
STEP_CONFIG = {
    "identify_customer": {
        "tools": [lookup_customer],
        "system_prompt": "고객을 식별하세요. 이메일 또는 계정 ID를 요청하세요.",
    },
    "diagnose_issue": {
        "tools": [check_service_status, escalate_to_resolve],
        "system_prompt": "이슈를 진단하세요. 도구를 사용한 후 escalate_to_resolve를 호출하세요.",
    },
}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 12

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

03_multi_agent_handoffs_router.ipynb

cell 19

```
STEP_CONFIG["resolve_issue"] = {
    "tools": [apply_fix, mark_resolved],
    "system_prompt": "이슈를 해결하세요. 수정을 적용한 후 mark_resolved를 호출하세요.",
}
STEP_CONFIG["close_ticket"] = {
    "tools": [send_satisfaction_survey, close_ticket],
    "system_prompt": "고객에게 감사를 전하고, 설문을 보내고, 티켓을 종료하세요.",
}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 13

이 코드는 미들웨어 훅과 실행 정책 삽입 지점을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 20

```
from typing import Callable
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse

@wrap_model_call
def step_middleware(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) → ModelResponse:
    """current_step 에 따라 system_prompt 와 tools 를 단일 호출 한정으로 교체."""
    step = request.state.get("current_step", "identify_customer")
    cfg = STEP_CONFIG[step]
    request = request.override(
        system_prompt=cfg["system_prompt"],
        tools=cfg["tools"],
    )
    return handler(request)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 14

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

03_multi_agent_handoffs_router.ipynb cell 22

```
from langchain.agents import create_agent

all_tools = [
    lookup_customer, check_service_status,
    escalate_to_resolve, apply_fix, mark_resolved,
    send_satisfaction_survey, close_ticket,
]
support_agent = create_agent(
    model="gpt-5.4", tools=all_tools,
    state_schema=SupportState, middleware=[step_middleware],
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 15

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 23

```
# [identify_customer]
#   User: "Can't log in. Email: alice@example.com"
#   Agent → lookup_customer("alice@example.com")
#         ← Command(update={current_step: "diagnose_issue"})
#
# [diagnose_issue] (auto transition)
#   Agent → check_service_status("auth-service")
#   Agent → escalate_to_resolve("Locked out")
#
# [resolve_issue] → [close_ticket]
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 16

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

03_multi_agent_handoffs_router.ipynb

cell 27

```
from pydantic import BaseModel, Field
from typing import Literal

class SubQuery(BaseModel):
    """소스별 하위 쿼리."""
    source: Literal["github", "notion", "slack"] = Field(description="지식 소스.")
    query: str = Field(description="해당 소스에 최적화된 검색 쿼리.")

class QueryClassification(BaseModel):
    """사용자 쿼리의 분류 결과."""
    sources: list[Literal["github", "notion", "slack"]] = Field(
        description="관련 지식 소스."
    )
    reasoning: str = Field(description="해당 소스를 선택한 이유.")
    sub_queries: list[SubQuery] = Field(description="소스별 하위 쿼리.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 17

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

03_multi_agent_handoffs_router.ipynb cell 28

```
from langchain.agents import AgentState

class RouterState(AgentState):
    classification: QueryClassification = None
    sources: list[str] = []
    sub_queries: list[SubQuery] = []
    agent_results: list[dict] = []
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 18

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

03_multi_agent_handoffs_router.ipynb cell 30

```
from langchain_openai import ChatOpenAI

classifier = ChatOpenAI(model="gpt-5.4").with_structured_output(
    QueryClassification
)

def route_query(state):
    """쿼리를 분류하고 라우팅을 결정합니다."""
    msg = state["messages"][-1].content
    cls = classifier.invoke(f"분류하고 하위 쿼리를 생성하세요:\n\n{msg}", config=lf_config)
    sq_dict = {sq.source: sq.query for sq in cls.sub_queries}
    return {"classification": cls, "sources": cls.sources, "sub_queries": cls.sub_queries}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 19

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

03_multi_agent_handoffs_router.ipynb cell 32

```
from langchain_core.tools import tool
from langchain.agents import create_agent

@tool
def search_github_code(query: str) → str:
    """GitHub 저장소를 검색합니다."""
    return f"'{query}'에 대한 GitHub 결과"

@tool
def search_notion_pages(query: str) → str:
    """Notion 워크스페이스를 검색합니다."""
    return f"'{query}'에 대한 Notion 결과"
```

03_multi_agent_handoffs_router 코드 20

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

03_multi_agent_handoffs_router.ipynb

cell 33

```python
@tool
def search_slack_messages(query: str) -> str:
    """Slack 메시지를 검색합니다."""
    return f"'{query}'에 대한 Slack 결과"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 21

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 34

```
github_agent = create_agent(
    model="gpt-5.4", tools=[search_github_code],
    system_prompt="GitHub에서 코드와 PR을 검색합니다.",
    name="github_agent",
)
notion_agent = create_agent(
    model="gpt-5.4", tools=[search_notion_pages],
    system_prompt="Notion에서 문서를 검색합니다.",
    name="notion_agent",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 22

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 35

```
slack_agent = create_agent(
    model="gpt-5.4", tools=[search_slack_messages],
    system_prompt="Slack에서 토론을 검색합니다.",
    name="slack_agent",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 23

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 36

```
from langgraph.types import Command, Send

def dispatch_to_agents(state):
    """하위 쿼리를 에이전트들에게 병렬로 전달합니다 (Send fan-out)."""
    cls = state["classification"]
    sq_dict = {sq.source: sq.query for sq in cls.sub_queries}
    return [
        Send(
            src,
            {
                "messages": [{"role": "user", "content": sq_dict.get(src, "")}],
                "source": src,
            },
        )
        for src in cls.sources
    ]
```

03_multi_agent_handoffs_router 코드 24

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

03_multi_agent_handoffs_router.ipynb

cell 38

```
def reduce_results(state):
    """모든 에이전트의 결과를 집계합니다."""
    results = state.get("agent_results", [])
    formatted = "\n".join(f"[{r['source']}] {r['content']}" for r in results)
    prompt = f"일관된 답변으로 합성하세요. 출처를 인용하세요.\n\n{formatted}"
    resp = ChatOpenAI(model="gpt-5.4").invoke(prompt, config=lf_config)
    return {"messages": [{"role": "assistant", "content": resp.content}]}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 25

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

03_multi_agent_handoffs_router.ipynb

cell 39

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(RouterState)
graph.add_node("router", route_query)
graph.add_node("github", github_agent)
graph.add_node("notion", notion_agent)
graph.add_node("slack", slack_agent)
graph.add_node("reducer", reduce_results)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 26

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

03_multi_agent_handoffs_router.ipynb

cell 40

```
graph.add_edge(START, "router")
graph.add_conditional_edges("router", dispatch_to_agents)
graph.add_edge("github", "reducer")
graph.add_edge("notion", "reducer")
graph.add_edge("slack", "reducer")
graph.add_edge("reducer", END)

app = graph.compile()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_multi_agent_handoffs_router 코드 27

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

03_multi_agent_handoffs_router.ipynb

cell 41

```
response = app.invoke({
    "messages": [{"role": "user",
        "content": "결제 서비스를 어떻게 배포하나요?"}]
}, config=lf_config)
print(response["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

04_context_memory.ipynb

04_context_memory.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 05_advanced/04_context_memory.ipynb

04_context_memory 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

04_context_memory.ipynb cell 4

```python
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

model = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("환경 준비 완료.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

04_context_memory.ipynb cell 5

```python
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON \u2014 project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

04_context_memory 코드 03

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

04_context_memory.ipynb

cell 8

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent

@dataclass
class UserContext:
    user_id: str
    role: str
    department: str
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 04

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 9

```python
@tool
def get_permissions(runtime: ToolRuntime[UserContext]) -> str:
    """현재 사용자의 역할에 따른 권한을 조회합니다."""
    ctx = runtime.context
    perms = {"admin": "read,write,delete", "editor": "read,write"}
    return f"사용자 {ctx.user_id} ({ctx.department}): {perms.get(ctx.role, 'read')}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 05

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

04_context_memory.ipynb

cell 10

```
agent = create_agent(
    model=model, tools=[get_permissions], context_schema=UserContext,
)
result = agent.invoke(
    {"messages": [{"role": "user", "content": "제 권한이 어떻게 되나요?"}]},
    context=UserContext(user_id="u42", role="admin", department="eng"),
    config=lf_config,
)
print(result["messages"][-1].content)
```

04_context_memory 코드 06

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

04_context_memory.ipynb cell 13

```python
from langchain.agents import AgentState

class RAGState(AgentState):
    """동적 검색 컨텍스트를 포함한 상태."""
    retrieved_docs: list[str]
    query_count: int

print(f"상태 키: {list(RAGState.__annotations__.keys())}")
```

04_context_memory 코드 07

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

04_context_memory.ipynb

cell 14

```
agent_with_state = create_agent(
    model=model, tools=[get_permissions],
    state_schema=RAGState, context_schema=UserContext,
)
result = agent_with_state.invoke(
    {"messages": [{"role": "user", "content": "안녕하세요!"}],
     "retrieved_docs": [], "query_count": 0},
    context=UserContext(user_id="u1", role="editor", department="sales"),
    config=lf_config,
)
print(f"최종 상태 키: {list(result.keys())}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 08

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 17

```python
# execution_info / server_info 노출 패턴
@tool
def whoami(runtime: ToolRuntime[UserContext]) -> str:
    """현재 run 의 identity 와 서버 메타데이터를 한 줄로 요약."""
    info = runtime.execution_info
    parts = [
        f"thread={info.thread_id}",
        f"run={info.run_id}",
        f"attempt={info.node_attempt}",
    ]
    server = runtime.server_info
    if server is not None:
        parts.append(f"assistant={server.assistant_id}")
        parts.append(f"graph={server.graph_id}")
        if server.user is not None:
            parts.append(f"user={server.user.identity}")
    else:
        parts.append("env=local")
    return " | ".join(parts)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 09

이 코드는 장기 메모리 Store 또는 저장 계층 사용을 보여줍니다.

04_context_memory.ipynb

cell 19

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
user_id = "user_42"
store.put((user_id, "preferences"), "theme", {"value": "dark"})
store.put((user_id, "preferences"), "language", {"value": "ko"})

item = store.get((user_id, "preferences"), "theme")
print(f"테마: {item.value}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 10

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 20

```
items = store.search((user_id, "preferences"))
for item in items:
    print(f"  [{item.key}] = {item.value}")

filtered = store.search(
    (user_id, "preferences"), filter={"value": "dark"}
)
print(f"필터 결과: {len(filtered)}건")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 11

이 코드는 장기 메모리 Store 또는 저장 계층 사용을 보여줍니다.

04_context_memory.ipynb

cell 22

```
semantic_store = InMemoryStore(
    index={"embed": embeddings, "dims": 1536}
)
ns = ("user_42", "memories")
semantic_store.put(ns, "mem1", {"content": "pytest를 unittest보다 선호"})
semantic_store.put(ns, "mem2", {"content": "모든 함수에 타입 힌트 사용"})
semantic_store.put(ns, "mem3", {"content": "좋아하는 음식은 초밥"})
semantic_store.put(ns, "mem4", {"content": "ML 인프라 팀에서 근무"})
print("임베딩과 함께 메모리 4개 저장 완료.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 12

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 23

```
results = semantic_store.search(
    ("user_42", "memories"), query="testing preferences", limit=2
)
for r in results:
    print(f"  [{r.key}] {r.value['content']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 13

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 24

```
results2 = semantic_store.search(
    ("user_42", "memories"), query="machine learning work", limit=2
)
for r in results2:
    print(f"  [{r.key}] {r.value['content']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 14

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 27

```python
@tool
def get_user_info(runtime: ToolRuntime[UserContext]) -> str:
    """현재 사용자의 저장된 정보를 조회합니다."""
    store = runtime.store
    user_id = runtime.context.user_id
    info = store.get(("users",), user_id)
    return str(info.value) if info else "사용자 정보를 찾을 수 없습니다."
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 15

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 28

```python
@tool
def save_preference(key: str, value: str, runtime: ToolRuntime[UserContext]) -> str:
    """사용자 선호도를 저장합니다."""
    store = runtime.store
    user_id = runtime.context.user_id
    store.put((user_id, "preferences"), key, {"value": value})
    return f"선호도 저장됨: {key}={value}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 16

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

04_context_memory.ipynb

cell 29

```
memory_agent = create_agent(
    model=model,
    tools=[get_user_info, save_preference],
    context_schema=UserContext,
    store=semantic_store,
)
result = memory_agent.invoke(
    {"messages": [{"role": "user", "content": "테마를 다크로 저장해 주세요"}]},
    context=UserContext(user_id="u42", role="admin", department="eng"),
    config=lf_config,
)
print(result["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 17

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

04_context_memory.ipynb

cell 30

```python
# TypedDict 입력을 store 에 저장할 때 dict() 캐스트 — runtime.context.user_id 를 namespace 에 사용
from typing_extensions import TypedDict

class UserInfo(TypedDict):
    name: str
    language: str

@tool
def save_user_info(user_info: UserInfo, runtime: ToolRuntime[UserContext]) -> str:
    """사용자 프로필을 저장합니다. TypedDict → dict 캐스트 필수."""
    assert runtime.store is not None
    runtime.store.put(("users",), runtime.context.user_id, dict(user_info))
    return f"saved user_info for {runtime.context.user_id}"

@tool
def get_user_info_typed(runtime: ToolRuntime[UserContext]) -> str:
    """저장된 프로필을 조회합니다."""
    assert runtime.store is not None
    item = runtime.store.get(("users",), runtime.context.user_id)
    return str(item.value) if item else "Unknown user"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 18

이 코드는 장기 메모리 Store 또는 저장 계층 사용을 보여줍니다.

04_context_memory.ipynb

cell 32

```
mem_store = InMemoryStore(index={"embed": embeddings, "dims": 1536})
uid = "user_42"

# Semantic -- Profile (single JSON)
mem_store.put((uid, "profile"), "main", {
    "name": "Alice", "language": "Python",
    "preferred_style": "concise",
})
# Semantic -- Collection (multiple docs)
mem_store.put((uid, "facts"), "f1", {"content": "pytest 선호"})
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 19

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 33

```
# Episodic -- past experiences (few-shot)
mem_store.put((uid, "episodes"), "ep1", {
    "content": "SQL 최적화 → EXPLAIN ANALYZE 사용",
})

# Procedural -- rules/guidelines
mem_store.put((uid, "procedures"), "rules", {
    "content": "항상 에러 처리를 포함. logging 사용.",
})
print("3가지 메모리 유형 모두 저장 완료.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 20

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 34

```python
# Episodic search: find similar past experiences
episodes = mem_store.search(
    (uid, "episodes"), query="database query help", limit=1
)
for ep in episodes:
    print(f"관련 에피소드: {ep.value['content']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 21

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

04_context_memory.ipynb

cell 36

```
skills = [
    {"name": "db_overview",
     "description": "모든 테이블의 고수준 개요",
     "content": "테이블: customers, orders, products"},
    {"name": "customers_schema",
     "description": "customers 테이블의 전체 스키마",
     "content": "CREATE TABLE customers (id INT PK, name VARCHAR)"},
]
SKILL_MAP = {s["name"]: s for s in skills}
print(f"스킬 {len(skills)}개 정의됨.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 22

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

04_context_memory.ipynb cell 37

```python
from langchain_core.tools import tool

@tool
def load_skill(skill_name: str) -> str:
    """데이터베이스 스킬에 대한 상세 정보를 로드합니다."""
    skill = SKILL_MAP.get(skill_name)
    if skill is None:
        return f"찾을 수 없음. 사용 가능: {', '.join(SKILL_MAP.keys())}"
    return f"## {skill['name']}\n\n{skill['content']}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 23

이 코드는 장기 메모리 Store 또는 저장 계층 사용을 보여줍니다.

04_context_memory.ipynb

cell 40

```python
from langgraph.store.base import BaseStore

# Hot path: write inline (adds latency)
def reflect_node(state, store: BaseStore):
    """메모리를 인라인으로 추출하고 저장합니다."""
    last_msg = state["messages"][-1].content
    store.put(("user", "reflections"), "latest", {"content": last_msg})
    return state

print("Hot path: 즉시 저장, 다음 턴에 사용 가능.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_context_memory 코드 24

이 코드는 장기 메모리 Store 또는 저장 계층 사용을 보여줍니다.

04_context_memory.ipynb

cell 41

```
import asyncio

# Background: write in separate async task
async def background_memory_writer(state, store: BaseStore):
    """백그라운드에서 메모리를 저장합니다 (지연 없음)."""
    last_msg = state["messages"][-1].content
    await store.aput(
        ("user", "reflections"), "latest", {"content": last_msg}
    )
print("Background: 최종 일관성, 지연 없음.")
```

CODE

05_agentic_rag.ipynb

05_agentic_rag.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 05_advanced/05_agentic_rag.ipynb

05_agentic_rag 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agentic_rag.ipynb cell 4

```python
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

llm = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("환경 준비 완료.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agentic_rag.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON \u2014 project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

05_agentic_rag 코드 03

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agentic_rag.ipynb

cell 8

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

raw_docs = [
    Document(page_content="LangGraph는 LLM으로 상태 기반 멀티 액터 "
        "애플리케이션을 구축하기 위한 프레임워크입니다.",
        metadata={"source": "langgraph-docs"}),
    Document(page_content="에이전트는 도구를 사용하여 외부 시스템과 "
        "상호작용합니다. ReAct 패턴은 추론과 행동을 번갈아 수행합니다.",
        metadata={"source": "agent-guide"}),
]
print(f"문서 {len(raw_docs)}개 로드됨.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 04

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 9

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200,
)
splits = text_splitter.split_documents(raw_docs)

for i, doc in enumerate(splits):
    print(f"청크 {i}: {doc.page_content[:60]}...")
print(f"총 청크 수: {len(splits)}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 05

이 코드는 장기 메모리 Store 또는 저장 계층 사용을 보여줍니다.

05_agentic_rag.ipynb

cell 11

```
from langchain_core.vectorstores import InMemoryVectorStore

vector_store = InMemoryVectorStore.from_documents(
    documents=splits, embedding=embeddings,
)
test_results = vector_store.similarity_search("LangGraph", k=2)
for doc in test_results:
    print(f"  [{doc.metadata['source']}] {doc.page_content[:80]}")
print(f"벡터 스토어 준비 완료. 문서 {len(splits)}개.")
```

05_agentic_rag 코드 06

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agentic_rag.ipynb

cell 13

```python
from langchain_core.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    """지식 베이스에서 관련 문서를 검색합니다."""
    docs = vector_store.similarity_search(query, k=4)
    serialized = "\n\n".join(
        f"출처: {d.metadata.get('source', '?')}\n{d.page_content}"
        for d in docs
    )
    return serialized, docs
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 07

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

05_agentic_rag.ipynb

cell 15

```
from langchain.agents import create_agent

rag_agent = create_agent(
    model=llm, tools=[retrieve],
    system_prompt="당신은 리서치 어시스턴트입니다. "
    "답변하기 전에 retrieve를 사용하여 검색하세요.",
)
response = rag_agent.invoke(
    {"messages": [{"role": "user", "content": "LangGraph란 무엇인가요?"}]},
    config=lf_config,
)
print(response["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 08

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agentic_rag.ipynb

cell 17

```python
from langchain.agents.middleware import dynamic_prompt

@dynamic_prompt
def rag_prompt(request):
    """문서를 검색하여 시스템 프롬프트에 주입합니다."""
    user_msg = request.state["messages"][-1].content
    docs = vector_store.similarity_search(user_msg, k=4)
    ctx = "\n\n".join(d.page_content for d in docs)
    return f"컨텍스트를 기반으로 답변하세요:\n\n{ctx}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 09

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

05_agentic_rag.ipynb

cell 18

```
chain = create_agent(model=llm, middleware=[rag_prompt])
resp = chain.invoke(
    {"messages": [{"role": "user", "content": "에이전트는 어떻게 작동하나요?"}]},
    config=lf_config,
)
print(resp["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 10

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agentic_rag.ipynb

cell 20

```python
from langgraph.graph import MessagesState

class AgentState(MessagesState):
    """커스텀 RAG 에이전트 상태."""
    relevance: str  # "relevant" or "not_relevant"

print(f"AgentState 키: {list(AgentState.__annotations__)}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 11

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 22

```
llm_with_tools = llm.bind_tools([retrieve])

def generate_query_or_respond(state: AgentState):
    """검색할지 직접 응답할지 결정합니다."""
    system = ("당신은 유용한 어시스턴트입니다. "
              "지식 베이스 관련 질문에는 retrieve 도구를 사용하세요.")
    msgs = [{"role": "system", "content": system}] + state["messages"]
    response = llm_with_tools.invoke(msgs, config=lf_config)
    return {"messages": [response]}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 12

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agentic_rag.ipynb

cell 24

```
from pydantic import BaseModel, Field
from typing import Literal

class GradeDocuments(BaseModel):
    """검색된 문서의 이진 관련성 점수."""
    relevance: Literal["relevant", "not_relevant"] = Field(
        description="문서가 관련이 있는지 여부."
    )
    reasoning: str = Field(description="간략한 설명.")

grader = llm.with_structured_output(GradeDocuments)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 13 (1/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 25

```
def grade_documents(state: AgentState):
    """
    검색된 문서의 관련성을 평가합니다.
    """

    msgs = state["messages"]

    user_q = next(
        (m.content for m in msgs if m.type == "human"),
        ""
    )

    tool_content = msgs[-1].content

    grade = grader.invoke(
        f"질문: {user_q}\n문서:\n{tool_content}\n"
        f"이 문서들이 관련이 있습니까?"
    )

    return {
        "relevance": grade.relevance,
        "messages": msgs
```

05_agentic_rag 코드 13 (2/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 25

```
}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 14 (1/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 27

```
def rewrite_question(state: AgentState):
    """
    검색 품질 향상을 위해 질문을 리라이트합니다.
    """

    user_q = next(
        (m.content for m in state["messages"] if m.type == "human"),
        ""
    )

    prompt = (
        f"다른 용어를 사용하여 리라이트하세요.\n"
        f"원본: {user_q}\n"
        f"리라이트:"
    )

    response = llm.invoke(prompt, config=lf_config)

    return {
        "messages": [
            {
                "role": "human",
```

05_agentic_rag 코드 14 (2/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 27

```
                "content": response.content
            }
        ]
    }
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 15 (1/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 29

```
def generate_answer(state: AgentState):
    """
    검색된 문서를 사용하여 답변을 생성합니다.
    """

    user_q = next(
        (m.content for m in state["messages"] if m.type == "human"),
        ""
    )

    docs = next(
        (m.content for m in state["messages"] if m.type == "tool"),
        ""
    )

    prompt = (
        f"컨텍스트만을 사용하여 답변하세요.\n"
        f"컨텍스트:\n{docs}\n\n"
        f"질문: {user_q}"
    )

    resp = llm.invoke(prompt, config=lf_config)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 15 (2/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 29

```
return {
    "messages": [
        {
            "role": "assistant",
            "content": resp.content
        }
    ]
}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 16

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

05_agentic_rag.ipynb

cell 31

```
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition

def relevance_router(state: AgentState):
    if state.get("relevance") == "relevant":
        return "generate_answer"
    return "rewrite_question"

graph = StateGraph(AgentState)
graph.add_node("gen_query", generate_query_or_respond)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 17

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 32

```
graph.add_node("retrieve", ToolNode([retrieve]))
graph.add_node("grade_documents", grade_documents)
graph.add_node("rewrite_question", rewrite_question)
graph.add_node("generate_answer", generate_answer)

graph.add_edge(START, "gen_query")
graph.add_conditional_edges(
    "gen_query", tools_condition,
    {"tools": "retrieve", "__end__": END},
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 18

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 33

```
graph.add_edge("retrieve", "grade_documents")
graph.add_conditional_edges(
    "grade_documents", relevance_router,
    {"generate_answer": "generate_answer",
     "rewrite_question": "rewrite_question"},
)
graph.add_edge("rewrite_question", "gen_query")
graph.add_edge("generate_answer", END)

app = graph.compile()
print("그래프 컴파일 성공.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agentic_rag 코드 19

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agentic_rag.ipynb

cell 34

```
for event in app.stream(
    {"messages": [{"role": "user", "content": "LangGraph란 무엇인가요?"}]},
    config=lf_config,
):
    for node_name, output in event.items():
        print(f"--- {node_name} ---")
        if "messages" in output:
            last = output["messages"][-1]
            txt = last.content if hasattr(last, "content") else str(last)
            print(txt[:200])
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

06_sql_agent.ipynb

06_sql_agent.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 05_advanced/06_sql_agent.ipynb

06_sql_agent 코드 01

이 코드는 SQL 데이터베이스 연결과 에이전트 도구화를 보여줍니다.

06_sql_agent.ipynb

cell 4

```
# %pip install langchain langchain-openai langchain-community langgraph sqlalchemy python-dotenv

from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
from langchain_community.utilities import SQLDatabase

llm = ChatOpenAI(model="gpt-5.4")
db = SQLDatabase.from_uri("sqlite:///Chinook.db")
print(f"Dialect: {db.dialect}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

06_sql_agent.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON \u2014 project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

06_sql_agent 코드 03

이 코드는 SQL 데이터베이스 연결과 에이전트 도구화를 보여줍니다.

06_sql_agent.ipynb

cell 8

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()

for t in tools:
    print(f"  {t.name}: {t.description[:60]}...")
print(f"총 도구 수: {len(tools)}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 04

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_sql_agent.ipynb

cell 10

```
system_prompt = (
    "당신은 SQL 에이전트입니다. 단계:\n"
    "1. sql_db_list_tables\n2. sql_db_schema\n"
    "3. 쿼리 작성 + sql_db_query_checker\n"
    "4. sql_db_query\n5. 결과를 해석하세요.\n"
    f"규칙: LIMIT 10 사용. DML 금지. Dialect: {db.dialect}"
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 05

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

06_sql_agent.ipynb

cell 11

```
from langchain.agents import create_agent

sql_agent = create_agent(
    model=llm, tools=tools, system_prompt=system_prompt,
)
print("LangChain SQL 에이전트 생성됨.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 06

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_sql_agent.ipynb

cell 13

```
response = sql_agent.invoke(
    {"messages": [{"role": "user",
     "content": "어떤 나라의 고객이 가장 많나요?"}]},
    config=lf_config,
)
print(response["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 07

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

06_sql_agent.ipynb cell 15

```python
from langchain.agents.middleware import HumanInTheLoopMiddleware

# DELETE / UPDATE 가 위험한 환경: sql_db_query 를 전부 가로채거나,
# 특정 도구만 allowed_decisions 로 제한할 수 있습니다.
hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "sql_db_query": {"allowed_decisions": ["approve", "edit", "reject"]},
    },
    description_prefix="SQL 쿼리 실행 승인 필요",
)
sql_agent_hitl = create_agent(
    model=llm, tools=tools,
    system_prompt=system_prompt, middleware=[hitl],
)
print("HITL이 적용된 SQL 에이전트 생성됨.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 08 (1/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

06_sql_agent.ipynb

cell 16

```
from langgraph.types import Command

config = {"configurable": {"thread_id": "sql-review-1"}}

# v2 invocation 필요 (version="v2")
# 1) 첫 실행 — interrupt 발생 시점까지 진행
# result = sql_agent_hitl.invoke(
#     {"messages": [{"role": "user", "content": "고객이 가장 많은 나라는?"}]},
#     config=config, version="v2",
# )
# print(result.interrupts)  # HITLRequest 확인

# Option 1: Approve
# result = sql_agent_hitl.invoke(
#     Command(resume={"decisions": [{"type": "approve"}]}),
#     config=config, version="v2",
# )

# Option 2: Edit
# result = sql_agent_hitl.invoke(
#     Command(resume={"decisions": [
#         {"type": "edit",
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 08 (2/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

06_sql_agent.ipynb

cell 16

```
#         "edited_action": {
#             "name": "sql_db_query",
#             "args": {"query": "SELECT Country, COUNT(*) FROM Customer GROUP BY Country LIMIT 10"},
#         }}
#     ]}),
#     config=config, version="v2",
# )

# Option 3: Reject (DELETE/UPDATE 차단 시 사용)
# result = sql_agent_hitl.invoke(
#     Command(resume={"decisions": [
#         {"type": "reject", "message": "DELETE 가 의심됩니다. SELECT 로 다시 작성하세요."}
#     ]}),
#     config=config, version="v2",
# )
print("HITL 재개 옵션: approve / edit / reject / respond (v2 invocation 필수)")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 09

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

06_sql_agent.ipynb

cell 18

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class SQLState(TypedDict):
    messages: Annotated[list, add_messages]

print(f"SQLState 키: {list(SQLState.__annotations__)}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 10

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_sql_agent.ipynb

cell 20

```
tool_map = {t.name: t for t in tools}

list_tbl = tool_map["sql_db_list_tables"]
schema_tl = tool_map["sql_db_schema"]
query_tl = tool_map["sql_db_query"]
check_tl = tool_map["sql_db_query_checker"]


def list_tables_node(state: SQLState):
    tables = list_tbl.invoke("", config=lf_config)

    msg = {
        "role": "assistant",
        "content": f"테이블: {tables}"
    }

    return {"messages": [msg]}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 11

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_sql_agent.ipynb

cell 21

```python
def get_schema_node(state: SQLState):
    """관련 테이블의 스키마를 가져옵니다."""
    resp = llm.invoke(state["messages"] + [
        {"role": "user",
         "content": "관련 테이블이 어떤 것인가요? 이름만 알려주세요."}
    ], config=lf_config)
    schema = schema_tl.invoke(resp.content.strip(), config=lf_config)
    msg = {"role": "assistant", "content": f"스키마:\n{schema}"}
    return {"messages": [msg]}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 12

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_sql_agent.ipynb

cell 23

```
llm_forced = llm.bind_tools(
    [check_tl], tool_choice="sql_db_query_checker"
)

def generate_query_node(state: SQLState):
    """SQL 쿼리를 생성합니다."""
    prompt = "SQL 쿼리를 작성해주세요. checker 도구를 사용하세요."
    msgs = state["messages"] + [{"role": "user", "content": prompt}]
    response = llm_forced.invoke(msgs, config=lf_config)
    return {"messages": [response]}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 13

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_sql_agent.ipynb

cell 24

```
def check_query_node(state: SQLState):
    """
    생성된 쿼리를 검증합니다.
    """

    last = state["messages"][-1]

    if hasattr(last, "tool_calls") and last.tool_calls:
        query = last.tool_calls[0]["args"].get("query", "")

        result = check_tl.invoke(query, config=lf_config)

        return {
            "messages": [
                {
                    "role": "tool",
                    "content": result
                }
            ]
        }

    return state
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 14 (1/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

06_sql_agent.ipynb cell 26

```python
from langgraph.types import interrupt

def execute_query_node(state: SQLState):
    """
    사람의 리뷰 후 쿼리를 실행합니다.
    """

    query = state["messages"][-1].content

    review = interrupt({
        "query": query,
        "action": "review_sql"
    })

    if review.get("action") == "accept":
        result = query_tl.invoke(query, config=lf_config)

    elif review.get("action") == "edit":
        result = query_tl.invoke(review["edited_query"], config=lf_config)

    else:
        result = f"거부됨: {review.get('reason', '')}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 14 (2/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

06_sql_agent.ipynb

cell 26

```
return {
    "messages": [
        {
            "role": "assistant",
            "content": result
        }
    ]
}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 15

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_sql_agent.ipynb

cell 28

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

builder = StateGraph(SQLState)
builder.add_node("list_tables", list_tables_node)
builder.add_node("get_schema", get_schema_node)
builder.add_node("generate_query", generate_query_node)
builder.add_node("check_query", check_query_node)
builder.add_node("execute_query", execute_query_node)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 16

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_sql_agent.ipynb

cell 29

```
builder.add_edge(START, "list_tables")
builder.add_edge("list_tables", "get_schema")
builder.add_edge("get_schema", "generate_query")
builder.add_edge("generate_query", "check_query")
builder.add_edge("check_query", "execute_query")
builder.add_edge("execute_query", END)

checkpointer = InMemorySaver()
sql_graph = builder.compile(checkpointer=checkpointer)
print("LangGraph SQL 에이전트 컴파일됨.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_sql_agent 코드 17

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

06_sql_agent.ipynb

cell 30

```python
from langgraph.types import Command

config = {"configurable": {"thread_id": "sql-1"}}

# Resume examples after interrupt:
# sql_graph.invoke(Command(resume={"action": "accept"}), {**config, **lf_config})
# sql_graph.invoke(Command(resume={
#     "action": "edit", "edited_query": "SELECT ..."
# }), {**config, **lf_config})
print("Command(resume=...) 패턴: accept / edit / reject")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

07_data_analysis.ipynb

07_data_analysis.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 05_advanced/07_data_analysis.ipynb

07_data_analysis 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

07_data_analysis.ipynb cell 4

```python
from dotenv import load_dotenv

load_dotenv()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

07_data_analysis.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON — project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

07_data_analysis 코드 03

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

07_data_analysis.ipynb

cell 8

```python
from deepagents.backends import LocalShellBackend

# 개발용 — 로컬 셸 백엔드
dev_backend = LocalShellBackend(virtual_mode=True)

# 운영용 — 클라우드 샌드박스 (택 1)
# prod_backend = ...  # 프로덕션에서는 클라우드 샌드박스 백엔드를 사용하세요
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 04

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

07_data_analysis.ipynb

cell 10

```
import os

workspace = "/tmp/analysis"
os.makedirs(workspace, exist_ok=True)

csv_content = (
    "region,quarter,revenue,units_sold\n"
    "Seoul,Q1,120000,340\nSeoul,Q2,135000,380\n"
    "Seoul,Q3,128000,355\nSeoul,Q4,150000,420\n"
    "Busan,Q1,85000,240\nBusan,Q2,92000,260\n"
    "Busan,Q3,88000,250\nBusan,Q4,105000,300"
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 05

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_data_analysis.ipynb

cell 11

```
with open(f"{workspace}/sales_2025.csv", "w") as f:
    f.write(csv_content)
print(f"CSV 저장됨: {workspace}/sales_2025.csv")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 06

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

07_data_analysis.ipynb

cell 13

```
from langchain_core.tools import tool
try:
    from slack_sdk import WebClient
except ImportError:
    WebClient = None
    print("slack_sdk 미설치 -- Slack 도구는 스텁으로 작동합니다")

slack_client = WebClient(token=os.environ.get("SLACK_BOT_TOKEN", "xoxb-placeholder")) if WebClient else None

@tool
def slack_send_message(message: str) → str:
    """분석 결과를 Slack 채널로 전송합니다."""
    resp = slack_client.chat_postMessage(
        channel=os.environ.get("SLACK_CHANNEL_ID", "general"), text=message)
    return f"전송 완료. 타임스탬프: {resp['ts']}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 07

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

07_data_analysis.ipynb

cell 15

```
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def tavily_search(query: str) -> str:
    """분석에 관련된 정보를 웹에서 검색합니다."""
    results = tavily_client.search(query, max_results=5)
    return "\n".join(
        [r["content"] for r in results["results"]]
    )
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 08

이 코드는 Deep Agent 생성과 하네스 옵션 연결을 보여줍니다.

07_data_analysis.ipynb

cell 17

```
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver

backend = LocalShellBackend(virtual_mode=True)
checkpointer = InMemorySaver()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 09

이 코드는 Deep Agent 생성과 하네스 옵션 연결을 보여줍니다.

07_data_analysis.ipynb

cell 18

```
agent = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=backend,
    checkpointer=checkpointer,
    system_prompt="당신은 데이터 분석가입니다.",
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 10

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_data_analysis.ipynb

cell 20

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "/tmp/analysis/sales_2025.csv의 매출 데이터를 분석하세요. 성과가 좋은 지역을 찾고, 분기별 트렌드를 파악하고, 시각화를 만들고, 요약을 Slack에 전송하세요."}]},
    config={**lf_config, "configurable": {"thread_id": "data-analysis-1"}},
)
print(result)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 11

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_data_analysis.ipynb

cell 22

```
for namespace, chunk in agent.stream(
    {"messages": [{"role": "user", "content": "지역별 매출을 요약해 주세요."}]},
    stream_mode="updates",
    subgraphs=True,
    config={**lf_config, "configurable": {"thread_id": "data-analysis-1"}},
):
    source = f"[subagent: {namespace}]" if namespace else "[main]"
    print(source, chunk)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 12

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_data_analysis.ipynb

cell 24

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "glob을 사용하여 /tmp/analysis/의 모든 CSV 파일을 나열하고, 첫 번째 파일을 읽어 구조를 설명해 주세요."}]},
    config={**lf_config, "configurable": {"thread_id": "data-analysis-1"}},
)
print(result)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 13

이 코드는 Deep Agent 생성과 하네스 옵션 연결을 보여줍니다.

07_data_analysis.ipynb

cell 26

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
config = {"configurable": {"thread_id": "analysis-session-1"}}

agent_with_memory = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=LocalShellBackend(virtual_mode=True),
    checkpointer=checkpointer,
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 14

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_data_analysis.ipynb

cell 27

```
r1 = agent_with_memory.invoke(
    {"messages": [{"role": "user", "content": "/tmp/analysis/sales_2025.csv를 읽고 요약해 주세요."}]},
    config={**config, **lf_config},
)
print("턴:", r1)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_data_analysis 코드 15

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_data_analysis.ipynb

cell 28

```
r2 = agent_with_memory.invoke(
    {"messages": [{"role": "user", "content": "서울과 부산의 Q4 실적을 비교해 주세요."}]},
    config={**config, **lf_config},
)
print("턴:", r2)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

08_voice_agent.ipynb

08_voice_agent.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 05_advanced/08_voice_agent.ipynb

08_voice_agent 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

08_voice_agent.ipynb

cell 4

```python
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_voice_agent 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

08_voice_agent.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON — project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

08_voice_agent 코드 03

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

08_voice_agent.ipynb

cell 10

```python
from langchain.agents import create_agent

def search_tool(query: str) -> str:
    """웹에서 최신 정보를 검색합니다."""
    return f"검색 결과: {query}"

def calendar_tool(action: str, details: str) -> str:
    """캘린더 이벤트를 관리합니다."""
    return f"캘린더 {action}: {details}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_voice_agent 코드 04

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

08_voice_agent.ipynb

cell 11

```
agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool],
    system_prompt=(
        "당신은 유용한 음성 어시스턴트입니다. "
        "응답을 간결하고 대화체로 유지하세요."
    ),
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_voice_agent 코드 05

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

08_voice_agent.ipynb

cell 13

```
async def stream_agent_response(user_text: str):
    """에이전트 응답 토큰을 하나씩 스트리밍합니다."""
    async for chunk in agent.astream(
        {"messages": [{"role": "user",
                       "content": user_text}]}
    ):
        if "messages" in chunk:
            for msg in chunk["messages"]:
                if hasattr(msg, "content") and msg.content:
                    yield msg.content
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_voice_agent 코드 06

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

08_voice_agent.ipynb

cell 19

```python
def weather_tool(city: str) -> str:
    """도시의 현재 날씨를 조회합니다."""
    return f"{city} 날씨: 15도, 구름 조금"

def reminder_tool(time: str, message: str) -> str:
    """지정된 시간에 알림을 설정합니다."""
    return f"{time}에 알림 설정됨: {message}"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_voice_agent 코드 07

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

08_voice_agent.ipynb

cell 20

```
voice_agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool,
           weather_tool, reminder_tool],
    system_prompt=(
        "당신은 음성 어시스턴트입니다. "
        "1-2문장으로 간결하고 대화체로 응답하세요."
    ),
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_voice_agent 코드 08

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

08_voice_agent.ipynb

cell 21

```
# 에이전트 단독 테스트 (텍스트 입력으로 동작 확인)
response = voice_agent.invoke(
    {"messages": [{"role": "user",
      "content": "서울 날씨가 어떤가요?"}]},
    config=lf_config,
)
print(response["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

09_production.ipynb

09_production.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 05_advanced/09_production.ipynb

09_production 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_production.ipynb

cell 4

```python
from dotenv import load_dotenv

load_dotenv()

import os
# Only enable LangSmith tracing if API key is available
if os.environ.get("LANGSMITH_API_KEY"):
    os.environ["LANGSMITH_TRACING"] = "true"
else:
    os.environ["LANGSMITH_TRACING"] = "false"
    print("LANGSMITH_API_KEY 미설정. LangSmith 트레이싱 비활성화.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_production 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_production.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON — project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_production 코드 03

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_production.ipynb cell 8

```
try:
    from langsmith import Client

    ls_client = Client()

    dataset = ls_client.create_dataset("agent-eval-v1")
    ls_client.create_examples(
        inputs=[{"query": "LangGraph란 무엇인가요?"}],
        outputs=[{"expected": "에이전트를 위한 프레임워크"}],
        dataset_id=dataset.id,
    )
    print("데이터셋 생성됨:", dataset.name)
except Exception as e:
    print(f"LangSmith 미설정 (건너뜀): {e}")
    ls_client = None
    dataset = None
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_production 코드 04

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_production.ipynb

cell 9

```
try:
    from agentevals.trajectory import create_trajectory_llm_as_judge

    evaluator = create_trajectory_llm_as_judge(
        rubric=(
            "에이전트가 적절한 도구를 사용했습니까? "
            "최종 답변이 정확했습니까?"
        ),
    )
    print("평가자 생성됨:", type(evaluator).__name__)
except ImportError:
    print("agentevals 미설치. 설치: pip install agentevals")
    evaluator = None
except Exception as e:
    print(f"평가자 생성 건너뜀 (LLM API 키 필요): {e}")
    evaluator = None
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_production 코드 05

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_production.ipynb

cell 11

```
from langchain_core.language_models import GenericFakeChatModel
from langchain_core.messages import AIMessage

fake_responses = [
    AIMessage(content="LangGraph는 프레임워크입니다."),
    AIMessage(content="상태 기반 에이전트를 지원합니다."),
]
fake_model = GenericFakeChatModel(
    messages=iter(fake_responses)
)
print(fake_model.invoke("LangGraph란 무엇인가요?", config=lf_config).content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_production 코드 06

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

09_production.ipynb

cell 12

```python
def search_tool(query: str) -> str:
    """웹에서 정보를 검색합니다."""
    return f"검색 결과: {query}"

def test_search_tool():
    """검색 도구가 예상 형식을 반환하는지 테스트합니다."""
    result = search_tool("test query")
    assert isinstance(result, str) and len(result) > 0
    print("통과: test_search_tool")

test_search_tool()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_production 코드 07

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_production.ipynb

cell 15

```
print("LANGSMITH_TRACING:", os.environ.get("LANGSMITH_TRACING"))
print("LANGSMITH_PROJECT:", os.environ.get("LANGSMITH_PROJECT"))

try:
    from langsmith import tracing_context

    with tracing_context(project_name="debug-session"):
        print("이 블록에 대해 트레이싱 활성화됨")
except Exception as e:
    print(f"LangSmith 트레이싱 사용 불가: {e}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_production 코드 08

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_production.ipynb

cell 17

```
try:
    from langsmith import tracing_context

    with tracing_context(
        project_name="production-agent",
        tags=["v2.1", "experiment-A"],
        metadata={"user_tier": "premium", "region": "kr"},
    ):
        print("태그된 트레이싱 활성화됨")
except Exception as e:
    print(f"LangSmith 트레이싱 사용 불가: {e}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_production 코드 09

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_production.ipynb

cell 21

```
import json

langgraph_config = {
    "dependencies": ["."],
    "graphs": {"agent": "./src/agent.py:graph"},
    "env": ".env",
}
print(json.dumps(langgraph_config, indent=2))
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_production 코드 10

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_production.ipynb

cell 24

```
try:
    from langgraph_sdk import get_sync_client

    api_key = os.environ.get("LANGSMITH_API_KEY", "")
    if not api_key:
        print("LANGSMITH_API_KEY 미설정. 클라이언트 연결 건너뜀.")
    else:
        client = get_sync_client(
            url="https://your-deployment.langsmith.com",
            api_key=api_key,
        )
        print("클라이언트 연결됨:", type(client).__name__)
except Exception as e:
    print(f"LangGraph SDK 클라이언트 사용 불가: {e}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CLOSING

고급 에이전트는 실행 흐름과 상태를 설계할 때 운영 가능해집니다

AI 에이전트 개발 고급. Middleware · Multi-agent · Memory · Production